Application Note

How to Implement Custom Serial Interfaces Using the Configurable Logic Block (CLB)

Gustavo Martinez

摘要

本应用报告介绍如何使用可配置逻辑块（CLB）设计自定义串行接口，并逐步说明 CLB 串行端口的设计方法、常见设计难点及潜在解决方案。最后，报告通过两个串行端口设计示例提供完整的设计细节、仿真结果和硬件测试结果。第一个示例介绍如何设计基于 CLB 的串行端口，以便通过音频系统中常用的时分复用（TDM）总线收发数据。第二个示例使用基于 CLB 的串行端口向 LED 点阵显示屏发送数据。所有示例代码均包含在 C2000Ware 软件开发套件（SDK）中。

目录

1 引言 • 3
2 串行端口设计方法 • 3
2.1 步骤 1：了解设计要求 • 3
2.2 步骤 2：确定 CLB Tile 所需输入 • 3
2.3 步骤 3：确定 CLB 逻辑输出 • 5
2.4 步骤 4：设计 CLB 逻辑 • 7
2.5 步骤 5：仿真逻辑设计 • 10
2.6 步骤 6：测试 CLB 逻辑 • 11
3 示例 A：使用 CLB 在音频应用中输入和输出 TDM 数据流 • 11
3.1 示例概述 • 11
3.2 步骤 1：了解设计要求 • 12
3.3 步骤 2：确定 CLB Tile 所需输入 • 12
3.4 步骤 3：确定 CLB 逻辑输出 • 13
3.5 步骤 4：设计 CLB 逻辑 • 14
3.6 步骤 5：仿真逻辑设计 • 18
3.7 步骤 6：测试 CLB 逻辑 • 19
4 示例 B：在照明应用中使用 CLB 为 LED 驱动器实现自定义通信总线 • 24
4.1 示例概述 • 24
4.2 步骤 1：了解设计要求 • 25
4.3 步骤 2：确定 CLB Tile 所需输入 • 26
4.4 步骤 3：确定 CLB 逻辑输出 • 27
4.5 步骤 4：设计 CLB 逻辑 • 28
4.6 步骤 5：仿真逻辑设计 • 35
4.7 步骤 6：测试 CLB 逻辑 • 38
5 参考资料 • 41

图目录

图 2-1：使用 D 型触发器同步输出 • 5
图 2-2：使用 FSM D 型触发器同步输出 • 6
图 2-3：D 型触发器的 FSM 配置 • 6
图 2-4：数据移动流程 • 8
图 2-5：互连模块示例框图 • 10
图 2-6：CLB 仿真示例 • 10
图 3-1：CLB TDM-8 示例 • 11
图 3-2：TDM 数据流所需帧同步脉冲、时钟信号和数据信号的时序 • 12

Copyright © 2023 Texas Instruments Incorporated

图 3-3. TDM-8 示例的 CLB Tile（TILE）输入.....12 图 3-4. TDM-8 示例的 CLB Tile（TILE）输出.....13 图 3-5. TDM-8 示例的 CLB Tile（TILE）逻辑.....14 图 3-6. CLB 逻辑设计所用的假定 TDM 数据流帧同步脉冲、时钟信号和数据信号时序.....14 图 3-7. TDM-8 示例的 FSM1 状态图.....15 图 3-8. TDM-8 示例的 FSM0 状态机.....16 图 3-9. TDM-8 数据接收仿真.....18 图 3-10. TDM-8 数据发送仿真.....18 图 3-11. FSYNC 和 DATA1 同步.....19 图 3-12. TDM 示例测试设置.....19 图 3-13. 导入 CLB TDM 示例项目.....21 图 3-14. 导入 McBSP TDM 示例项目.....22 图 3-15. 未使用锁存和延迟逻辑的 TDM-8 输出.....23 图 3-16. 最终 TDM-8 输出.....23 图 3-17. F28388D 监测输入的 TDM 流量.....24

表目录

表目录（续）

商标

C2000™ 和 Code Composer Studio™ 是 Texas Instruments 的商标。所有商标均归其各自所有者所有。

Copyright © 2023 Texas Instruments Incorporated

1 引言

串行接口广泛用于许多工业和汽车应用中。如今，微控制器提供的标准串行接口可用于访问各种外设。然而，在某些情况下，为支持特定串行协议，可能需要自定义串行接口。C2000™ 实时控制微控制器中集成的可配置逻辑块（CLB）可编程以模拟这些自定义串行接口。

本应用报告提供逐步设计方法，指导设计人员完成自定义串行接口的设计、仿真和测试。报告还介绍了常见设计难点及其潜在解决方案，最后给出两个包含完整设计细节、仿真结果和硬件测试结果的示例。示例代码可从 C2000Ware 软件开发套件（SDK）轻松获取。

2 串行端口设计方法

以下指南按步骤介绍如何使用 CLB 设计自定义串行接口，带领设计人员从头到尾完成 CLB 逻辑设计，并提供常见设计难题和易错点的解决方案。

本应用报告假定读者已熟悉 CLB 架构以及 Code Composer Studio（CCS）集成开发环境（IDE）。以下培训资料可帮助读者快速了解 CLB 和 C2000 实时控制微控制器：

- 可配置逻辑块（CLB）简介（视频）
- 可配置逻辑块（CLB）架构（视频）
- 可配置逻辑块（CLB）编程工具培训（视频）
- 面向 C2000 实时控制微控制器的 SysConfig 开发工具
- C2000 Academy 在线培训

不同 C2000 实时控制微控制器上的 CLB 类型、CLB Tile（TILE）数量和交叉开关（XBAR）逻辑可能不同。有关特定 C2000 实时控制微控制器的 CLB 详细信息，请参阅相应器件的 TRM。

2.1 步骤 1：了解设计要求

成功设计 CLB 逻辑的关键在于清晰理解设计要求（*any*）。对于串行接口设计，在 CLB 逻辑设计阶段必须全面了解总线协议，包括信号数量、输入和输出配置，以及帧编码要求，例如起始位、停止位、奇偶校验位和校验和位。

其次，必须明确了解总线接口的时序要求，以判断是否可用 CLB 实现该设计。应特别关注串行总线的运行速率、信号极性以及所需的建立时间和保持时间。

最后，应识别实现串行总线接口所需的所有辅助信号。例如，用于在 CLB Tile（TILE）中输入和输出数据的片上 PWM 生成的时钟信号，
或者用于触发在串行总线上发送同步命令的周期性定时器中断。

2.2 步骤 2：确定 CLB Tile（TILE）所需的输入

大多数情况下，需要将输入信号传入 CLB Tile（TILE）。通常应考虑以下三类信号：

- 通过 GPIO 引脚采样的外部串行总线信号，例如串行总线时钟、帧同步和数据信号。
- 用于 CLB 逻辑运行的片上外设信号，例如通过 PWM*n*A 引脚生成的时钟信号或定时器中断。
- 由 CPU 直接控制的 GPREG 位。可使用这些 GPREG 位触发 CLB Tile（TILE）中的操作，或启用/禁用特定功能。

可结合器件上的不同 XBAR，通过 CLB 全局和本地多路复用器将这些输入信号连接到 8 个 CLB Tile（TILE）输入。有关详细信息，请参阅相应器件的 TRM。

Copyright © 2023 Texas Instruments Incorporated

通常有多条路径可将输入信号连接到 CLB Tile（TILE）。表 2-1 列出了将这些信号引至 CLB Tile（TILE）边界的推荐路径。

表 2-1. 推荐的输入信号路径

Input Signal Type	Recommended Input Path
External serial bus signals (for example, serial bus clock, frame, and data signals)	GPIO pin ⇨ CLB input XBAR 1 ⇨ CLB local mux ⇨ CLB input
Internal on-chip peripheral signals (for example, clock signal generated using a PWM*n*A)	Peripheral signal ⇨ CLB global mux or CLB local mux ⇨ CLB input
Custom signal directly controlled by CPU	Memory-mapped GPREG bit ⇨ CLB input

1. 并非所有 C2000 实时控制微控制器都提供 CLB 输入 XBAR。在未提供该功能的器件上，可通过 GPIO XBAR 和 CLB XBAR 将外部信号引至 CLB Tile（TILE）边界。详情请参阅相应器件的 TRM。

2.2.1 GPIO 输入限定

采样外部串行总线信号时，一个重要的考虑因素是 GPIO 引脚的输入限定设置。表 2-2 汇总了可用的输入限定设置及其潜在用途。

表 2-2. GPIO 输入限定用途

Input Qualification Setting	Description	Uses
Asynchronous input (recommended setting)	The input signal is not synchronized to SYSCLKOUT. Note: Synchronization can be enabled at CLB input level (see Section 2.2.2).	Used for passing signals unchanged to the CLB tile.
Synchronization to SYSCLKOUT (not recommended)	The input signal is synchronized to SYSCLKOUT.	A small analog delay is added by the input XBARs before the signal reaches the CLB tile boundary, essentially making the signal asynchronous to SYSCLKOUT again. Therefore, synchronization at this level only adds unnecessary delay. It is recommended to enable synchronization at the CLB input level instead (see Section 2.2.2).
Qualification Using a Sampling Window (recommended in some cases)	The input signal is synchronized to SYSCLKOUT and qualified for a specified number of cycles before the input is allowed to change.	Use for removing noise from the input signal. However, the added latency introduced by the sampling window needs to be considered when designing CLB logic. Synchronization at the CLB input level should also be enabled to re-synchronize the signal to SYSCLKOUT since the input XBARs will add a small analog delay before the signal reaches the CLB tile boundary.

还应考虑以下功能：

- GPIO 内部上拉电阻：如果电路板上没有外部上拉电阻，应启用此功能，以避免 CLB 输入信号悬空。
- GPIO 输入信号反相：启用此功能可在信号送入 CLB 之前，对 GPIO 引脚处的信号进行反相。

Copyright © 2023 Texas Instruments Incorporated

2.2.2 CLB 输入设置

8 个 CLB 输入各自具有不同的配置和滤波选项；应根据串行总线的开关特性仔细选择。表 2-3 和表 2-4 列出了不同选项及其可能用途。

表 2-3. CLB 输入配置选项

Input Configuration Setting	Description	Uses
Asynchronous input	The input signal is not synchronized to SYSCLKOUT.	Use for passing signals unchanged through the CLB tile and directly to the CLB outputs.
Synchronization to SYSCLKOUT (recommended setting)	The input signal is synchronized to SYSCLKOUT.	Generally, it is required to synchronize all CLB inputs used in tile logic blocks.

表 2-4. CLB 输入滤波选项

Input Filter Setting	Description	Uses
No filtering	The input signal is passed directly to the CLB tile.	Use for designing logic that depends on the logical state of the input. For example, enable & disable signals.
Rising-edge detect	A single pulse equal to a CLB clock will be generated when a rising-edge is detected on the input signal.	Use for shifting data in or out of the counters in serializer mode or for counting serial clock edges.
Falling-edge detect	A single pulse equal to a CLB clock will be generated when a falling-edge is detected on the input signal.	Use for shifting data in or out of the counters in serializer mode or for counting serial clock edges.
Any-edge detect	A single pulse equal to a CLB clock will be generated when a falling-edge or rising-edge is detected on the input signal.	Use for logic that needs to act on both edges of an input signal.

有时需要将单个外部输入映射到多个 CLB Tile（TILE）输入，并为各输入设置不同的滤波选项。这样，部分 CLB Tile（TILE）逻辑可以由外部信号的上升沿触发，其他逻辑则由同一信号的下降沿触发。本应用报告的两个示例均采用了将单个外部信号映射到多个 CLB 输入的做法。

2.3 步骤 3：确定 CLB 逻辑所需的输出

大多数情况下，CLB Tile（TILE）需要驱动串行总线信号，例如时钟、帧同步和数据信号。每个 CLB Tile（TILE）有 8 个输出，可通过
CLB 输出 XBAR 或 GPIO 输出 XBAR 引出至器件引脚。

2.3.1 同步输出信号

CLB 生成的输出信号很可能来自 CLB Tile（TILE）中的不同逻辑块，因此各输出信号可能具有不同的延迟路径。为尽可能增大接收器件所见的建立时间和保持时间，可以使用总线时钟和简单触发器使不同 CLB 输出彼此同步，参见图 2-1。这些触发器可通过 CLB Tile（TILE）中的有限状态机（FSM）模块实现。不过，CLB Tile（TILE）资源有限，因此只有在确有需要时才建议采用此方案。

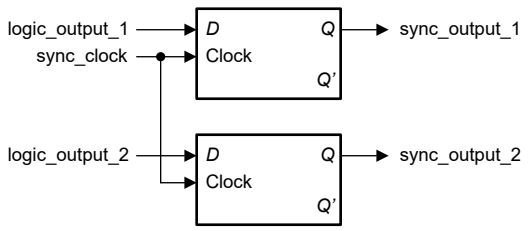


图 2-1. 使用 D 型触发器同步输出

Copyright © 2023 Texas Instruments Incorporated

图 2-2 所示仿真展示了两个输入 *in1* 和 *in2* 在略有不同的时刻改变状态的情况，可借此说明同步逻辑的必要性。使用由两个 FSM 实现的边沿触发 D 型触发器，并借助第三个“时钟”信号 *in0* 使这两个信号彼此同步。仿真结果显示，每个 FSM 都使用“时钟”信号 *in0* 锁存并延迟其输入信号。因此，两个 FSM 的输出彼此同步。

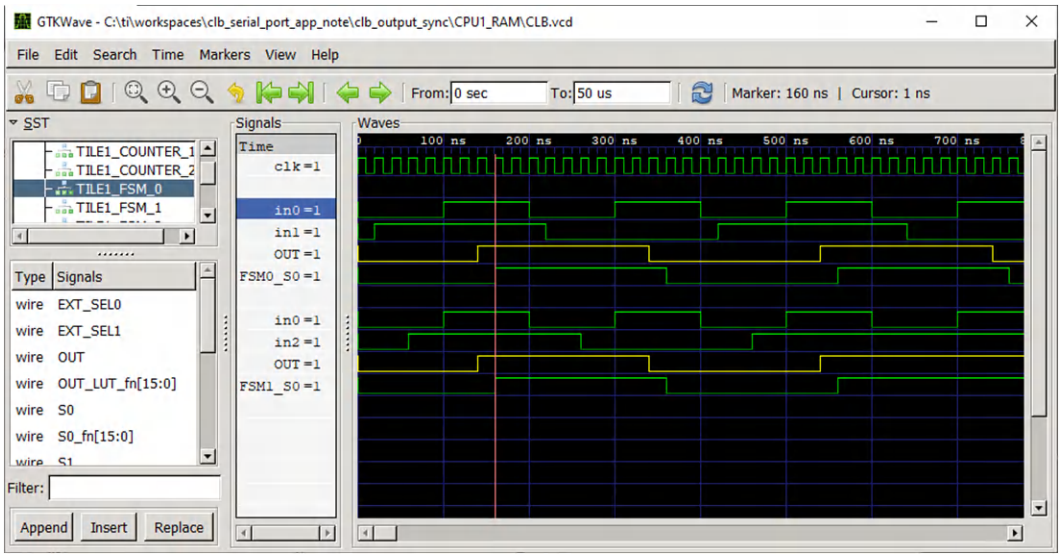


图 2-2. 使用 FSM D 型触发器同步输出

图 2-3 给出了每个 FSM 输出和状态变量的逻辑方程。注意，FSM *out* 和 *s0* 使用相同的方程，且可由任一信号驱动最终输出。由于 *out* 是纯组合输出，而 *s0* 始终在下一个 CLB 时钟周期更新，因此两个信号之间存在 1 个周期的延迟。另请注意，在此仿真中，*in0* 输入滤波器配置为同步上升沿检测。

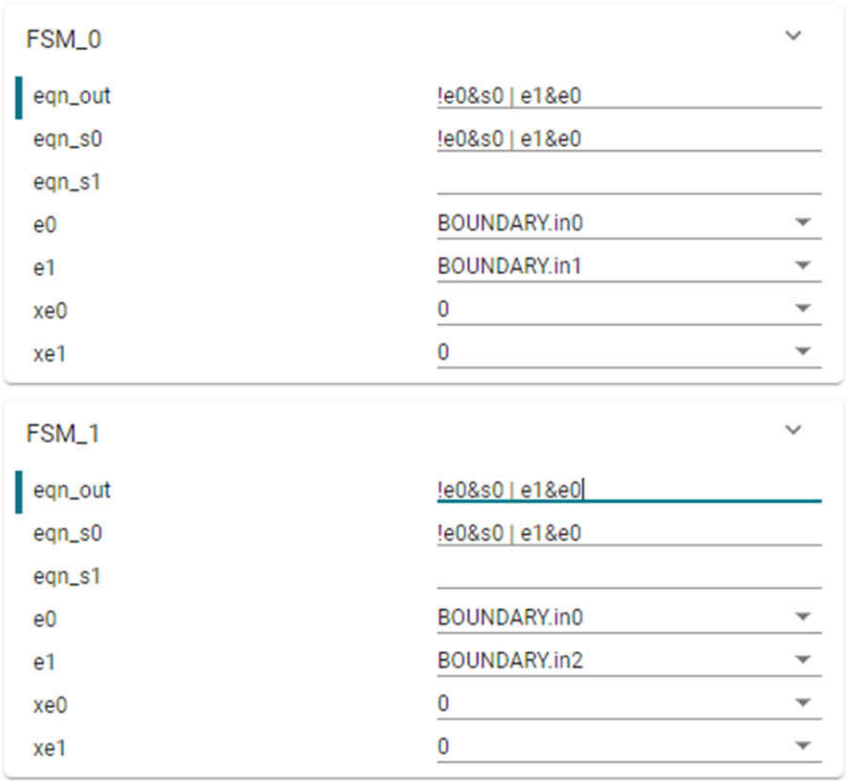


图 2-3. D 型触发器的 FSM 配置

Copyright © 2023 Texas Instruments Incorporated

2.3.2 输出信号调节

每个 CLB 输出都可以通过异步输出调节（AOC）模块，在送至输出 XBAR 之前进行反相和门控。也可利用 AOC 在信号送往输出端之前对其反相。

Note

If a CLB input signal, say a serial clock, needs to be passed through the CLB tile unsynchronized to the internal CLB clock, the input signal must be passed through the CLB tile using boundary input 4 or 5. From the tile input the signal must be passed directly to the AOC where it can be inverted, if desired. Additionally, the AOC output must be brought to the microncroller pin through the GPIO OUTPUT XBAR, not the CLB OUTPUT XBAR.

2.4 步骤 4：设计 CLB 逻辑

设计 CLB 逻辑是整个流程中最复杂的步骤。要完成设计，通常需要多次迭代逻辑设计、仿真和测试。

编程 CLB Tile（TILE）最简单的方法是使用 CLB 工具。借助该工具，用户可以为每个 CLB Tile（TILE）输入选择滤波选项、连接各 CLB 模块中的子模块、配置 FSM、高级控制器（HLC）和其他逻辑模块，还可以轻松地将一个 Tile 的逻辑复制到其他 Tile。该工具还支持为 Tile 逻辑设置仿真，并会实时指出逻辑设计中的潜在问题，例如不受支持的逻辑连接和无效的 FSM 公式。

CLB 工具使用 SysConfig 图形用户界面（GUI），该工具是 Code Composer Studio™（CCS）的一部分。用户还可通过 SysConfig 配置 MCU 的输入和输出 XBAR，并对 MCU 外设进行编程。

有关 CLB 工具和 SysConfig 的更多信息，请参阅以下资料：

- CLB 工具用户指南
- 可配置逻辑块（CLB）编程工具培训（视频）
- 面向 C2000 实时控制微控制器的 SysConfig 开发工具

2.4.1 资源分配

由于每个 CLB Tile（TILE）的资源有限，因此必须仔细规划 CLB 逻辑模块的资源使用。通常，串行接口至少需要以下资源：

- 一个计数器用于数据接收。该计数器以串行器模式运行，使 CLB 能够在每个串行时钟边沿移入一位数据。
- 一个计数器用于数据发送。该计数器以串行器模式运行，使 CLB 能够在每个串行时钟边沿移出一位数据。
- 一个计数器用于统计串行位数。该计数器以普通计数器模式运行。
- 一个 FSM 模块用于统计接收和/或发送的数据字，并向 HLC 触发事件。

虽然可以完全在一个 CLB Tile（TILE）中实现串行接口，但某些情况下可能需要使用两个 CLB Tile（TILE）。此时，建议将接收和发送功能分别放在两个 Tile 中，以简化逻辑设计。

某些串行接口需要起始位、停止位和奇偶校验位等特殊帧编码位。CLB 可以支持这些功能，但需要额外的 CLB 逻辑模块。如果没有可用的 CLB 模块，可考虑使用 C28x CPU 从接收数据中提取这些位，或将其添加到发送数据中。

2.4.2 在 CLB FIFO 与 MCU RAM 之间交换数据

基于 CLB 的串行接口设计需要在以下两处传送数据：1. CLB 串行器与 CLB FIFO 之间；2. CLB FIFO 与 MCU RAM 之间。CLB Tile（TILE）中的 HLC 可以在串行器与 FIFO 之间传送数据。不过，由于 HLC 在 MCU 架构中不是总线主控器，因此不能直接在 FIFO 与 MCU RAM 之间传送数据。撰写本应用报告时，只有 C28x CPU 和 CLA 能够作为主控器在 CLB FIFO 中读写数据，因此不能使用 DMA 完成这类数据传送。

Copyright © 2023 Texas Instruments Incorporated

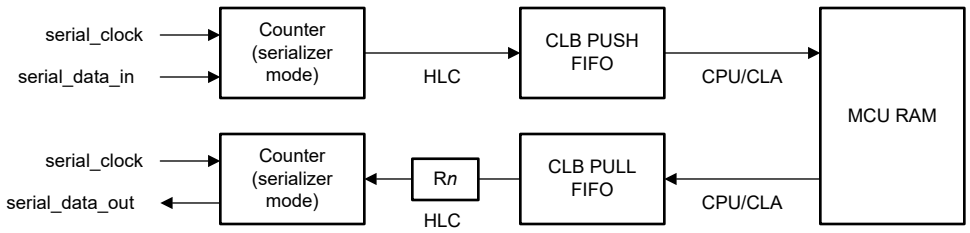


图 2-4. 数据移动流程

注

HLC 无法将数据直接读入 CLB 计数器。必须先将数据读入某个 HLC 寄存器（R*n*），再将数据移至计数器。

CPU 可以通过 CLB 本地写接口总线直接加载计数器和 HLC 寄存器。
这有助于设置 CLB Tile（TILE）的初始状态。例如，在第 4 节“在照明应用中使用 CLB 为 LED 驱动器实现自定义通信总线”中，启用 CLB TX Tile 进行发送前，会先将发送字总数载入计数器。以下代码展示了如何使用 C2000Ware DriverLib 函数实现此操作。有关 CLB 本地写接口总线的信息，请参阅相应器件的 TRM。

```
CLB_writeInterface(TX_TILE_BASE, CLB_ADDR_COUNTER_1_LOAD, 0xFFFFFFFF); CLB_write
Interface(TX_TILE_BASE, CLB_ADDR_HLC_R2, 0x0);
```

2.4.3 CLB 逻辑状态和触发标志

使用 CLB 设计串行接口时，通常需要实现状态/标志位和触发位。

2.4.3.1 Status/Flag Bits

状态/标志位可供 CPU 轮询 CLB 逻辑的状态。例如，CLB 产生中断后，CPU 可以轮询标志位，判断该中断是由数据接收事件还是接收错误事件引起。

这些状态/标志位可通过 HLC 寄存器实现。不过，每个标志都必须使用一个 HLC 寄存器来表示，因为 HLC 不支持按位 AND 或 OR 指令。例如，非零值
例如，R0 寄存器中的非零值可用于表示接收中断，而 R1 中的非零值可用于表示接收错误。

注

由于后续执行 HLC “INTR <tag>”指令会覆盖先前的标记值，因此不能使用该指令和 CLB_INTR_TAG_REG 实现标志位。

也可以直接读取各 CLB 逻辑模块的状态来判断 CLB 逻辑状态。例如，
如果将 FSM 配置为包含多个状态（例如 IDLE 和 ACTIVE），CPU 可读取 S0 和 S1 的状态来判断 CLB 逻辑状态。

CPU 可以读取 CLB 存储器映射调试寄存器 CLB_DBG_R*n*、CLB_DBG_C*n* 和 CLB_DBG_OUT，判断各 CLB 模块的状态。以下代码展示了如何使用 HLC 寄存器作为状态位。代码使用 C2000Ware DriverLib 函数。

```
uint32_t chkIntFlag = CLB_getRegister(CCSI2_RX_TILE_BASE, CLB_REG_HLC_R1); uint32_t rcvIntFlag
= CLB_getRegister(CCSI2_RX_TILE_BASE, CLB_REG_HLC_R2); uint32_t endIntFlag = CLB_getRegiste
r(CCSI2_RX_TILE_BASE, CLB_REG_HLC_R3);

...   // 接收中断   if (rcvIntFlag & 0x1)   {       ...       // 清除接收中断标志寄存器       CLB_wri
teInterface(CCSI2_RX_TILE_BASE, CLB_ADDR_HLC_R2, 0x0);   }
```


Copyright © 2023 Texas Instruments Incorporated

```
// 结束中断
if (endIntFlag & 0x1)
{
...
// 清除结束中断标志寄存器
CLB_writeInterface(CCSI2_RX_TILE_BASE, CLB_ADDR_HLC_R3, 0x0);
}

// 检查错误中断
if (chkIntFlag & 0x1)
{
...
// 清除检查错误中断标志寄存器
CLB_writeInterface(CCSI2_RX_TILE_BASE, CLB_ADDR_HLC_R1, 0x0);
}
```

2.4.3.2 Trigger Bits

触发位可供 CPU 启用或禁用 Tile 中的逻辑，或触发特定操作。例如，可以设置接收器使能位，在运行时有选择地启用或禁用 CLB 接收逻辑。这些触发位可通过 CLB_GP_REG 寄存器实现。

CLB_GP_REG 位可直接连接到 8 个 CLB Tile（TILE）输入中的任意输入。CPU 可以设置 CLB_GP_REG 中的任意位，以触发 CLB 逻辑操作。例如，可设置接收器使能/禁用位，在运行时启用或禁用 CLB 接收逻辑。以下代码展示了如何实现该使能/禁用位。代码使用 C2000Ware DriverLib。

```
#define GPREG_ENABLE_RCVR          3U void CCSI_HAL_enableClbReceiver() {    uint32_t gp
RegVal = CLB_getGPREG(CCSI1_RX_TILE_BASE);    // 首先检查接收器是否处于 IDLE 状态
while(HWREG(CCSI1_RX_TILE_BASE + CLB_LOGICCTL + CLB_O_DBG_OUT) &           (C
LB_DBG_OUT_FSM0_S1 | CLB_DBG_OUT_FSM0_S0)) {}    // 启用接收器    gpRegVal |= (1
U << GPREG_ENABLE_RCVR);    CLB_setGPREG(CCSI1_RX_TILE_BASE, gpRegVal); }void
CCSI_HAL_disableClbReceiver() {    uint32_t gpRegVal = CLB_getGPREG(CCSI1_RX_TILE_BA
SE);    // 首先检查接收器是否处于 IDLE 状态    while(HWREG(CCSI1_RX_TILE_BASE + CLB
_LOGICCTL + CLB_O_DBG_OUT) &           (CLB_DBG_OUT_FSM0_S1 | CLB_DBG_OUT_F
SM0_S0)) {}    // 禁用接收器    gpRegVal &= ~(1U << GPREG_ENABLE_RCVR);    CLB_set
GPREG(CCSI1_RX_TILE_BASE, gpRegVal); }
```

注

逻辑设计中使用的每个 CLB_GP_REG 位都必须专用一个 CLB 输入。CLB 输入最多只有 8 个，因此应谨慎使用 CLB_GP_REG 位。

Copyright © 2023 Texas Instruments Incorporated

2.5 步骤 5：仿真逻辑设计

由于设计中涉及不同的逻辑模块及其互连，串行接口设计很快就会变得复杂。为便于调试 CLB 逻辑，CLB 工具会生成互连框图（图 2-5）和仿真波形（图 2-6）。有关如何生成这两个文件的详细信息，请参阅 *CLB Tool User's Guide*。

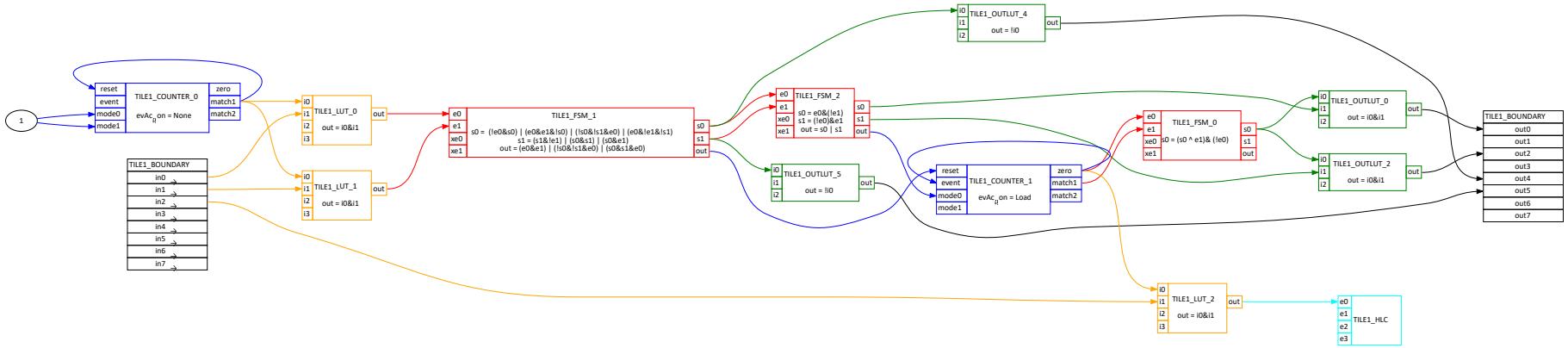


图 2-5. 互连框图示例

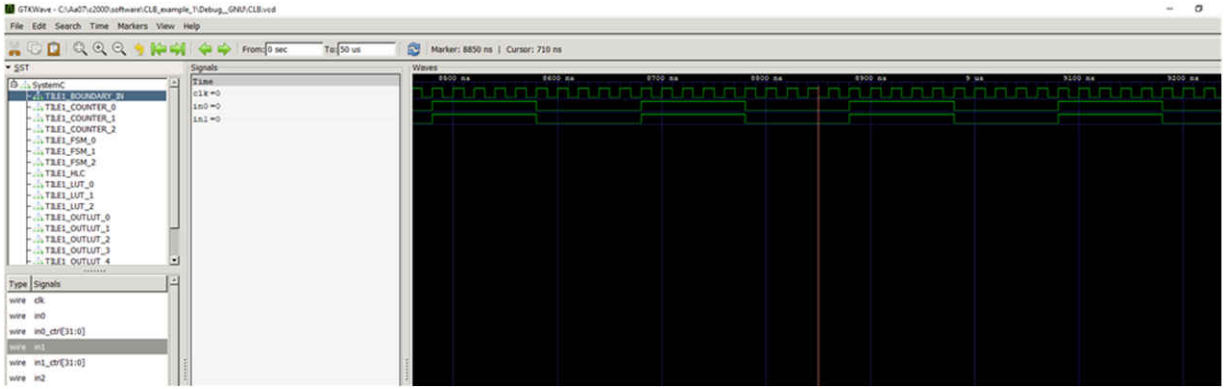


图 2-6. CLB 仿真示例

Copyright © 2023 Texas Instruments Incorporated

2.6 步骤 6：测试 CLB 逻辑

设计流程的最后一步是在目标系统上运行完整设计并进行测试。

建议先搭建受控测试环境，在将设计用于目标应用之前，全面验证逻辑设计。测试应尽可能覆盖 CLB 逻辑的所有功能，包括改变数据模式和时钟频率，并设置有意引入错误条件的机制；如果 CLB 逻辑用于检测错误，这一点尤其重要。

可以使用 CLB 仿真器调试测试中发现的问题，方法是调整仿真激励并查看 CLB 逻辑模块中的信号。修改 CLB 逻辑后，应重新编译并运行测试。

> 注意
> 请注意，在室温下对单个系统进行测试并不能保证系统在目标工作条件下均能正常运行。应识别所有关键设计参数，并确保测试覆盖充分、时序裕量充足，以保证逻辑设计能够满足这些参数要求。

3 示例 A：在音频应用中使用 CLB 输入和输出 TDM 数据流

3.1 示例概述

时分复用是音频应用中常用的一种方法，可在系统中的器件之间传输和接收音频数据。在这种复用方式中，一帧被划分为多个固定长度的时隙或通道。多个通道按时间分隔后复用到一个传输通道中，因此称为时分复用（TDM）。

部分 C2000 实时控制微控制器配备的多通道缓冲串行端口（McBSP）支持输入和输出单路 TDM 数据流。不过，并非所有 C2000 实时控制微控制器都提供 McBSP 外设。此外，C2000 实时控制微控制器通常配备的其他串行端口外设均不支持 TDM 协议。

本示例使用 CLB 输入 8 通道 TDM 数据流（TDM-8），并输出相应的 TDM-8 数据流。C28x CPU 在 CLB FIFO 与内部 RAM 之间搬运接收和待发送的数据。本示例使用一个 CLB Tile（TILE）实现，输入和输出 TDM-8 数据流均由外部 12.288 MHz 时钟驱动。

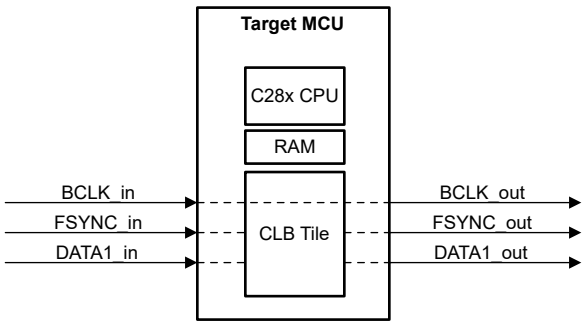


图 3-1. CLB TDM-8 示例

Copyright © 2023 Texas Instruments Incorporated

3.2 步骤 1：了解设计要求

CLB Tile（TILE）逻辑必须按照图 3-2 所示的时序，输入和输出 8 通道 TDM 数据流，包括时钟、帧同步和数据信号。

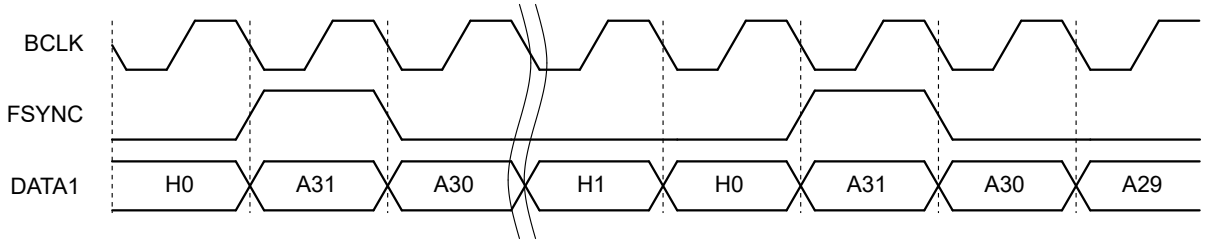


图 3-2. TDM 数据流所需的帧同步脉冲、时钟信号和数据信号时序

具体需要满足以下条件：

- 外部固定频率为 12.288MHz 的串行位时钟（位时钟直通至输出）
- 帧同步脉冲宽度为 1 个时钟周期，且无延迟
- 帧同步脉冲为高电平有效
- 在时钟上升沿采样发送/接收数据
- 每个 TDM 帧包含 8 个通道
- 每个通道的数据字长为 32 位

只需使用 GPIO 反相逻辑，在信号到达 CLB Tile（TILE）之前将其反相，即可轻松支持其他帧同步和时钟极性。

3.3 步骤 2：确定 CLB Tile（TILE）所需的输入

输入的 TDM-8 数据流信号通过 CLB 输入 XBAR 路由到 CLB Tile（TILE）边界输入，如图 3-3 所示。

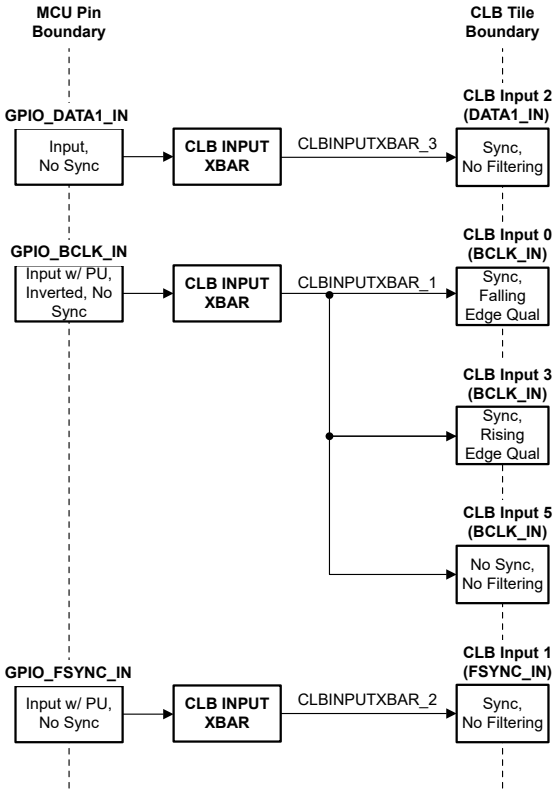


图 3-3. TDM-8 示例的 CLB Tile（TILE）输入

Copyright © 2023 Texas Instruments Incorporated

用于输入 TDM 数据流的所有 GPIO 均配置为异步运行。根据需要，在 CLB Tile（TILE）输入边界启用同步。此外，为避免输入引脚悬空，应启用 BCLK_IN 和 FSYNC_IN GPIO 的内部上拉。

注

CLB 逻辑采用的时钟极性与图 3-2 所示相反，因此 BCLK_IN 信号需要反相。详情请参阅第 3.5 节。

BCLK_IN 信号被路由到 3 个采用不同滤波配置的 Tile 输入。

- CLB Input 0：在 BCLK_IN 的下降沿触发数据接收。
- CLB Input 3：在 BCLK_IN 的上升沿触发数据发送。
- CLB Input 5：禁用此输入的滤波和同步，以满足 BCLK 直通要求。

必须特别考虑将 BCLK_IN 信号直通到 BCLK_OUT 输出的要求。若将输入的 12.288 MHz BCLK 信号同步到 100 MHz CLB 内部时钟，由于两个时钟频率不是整数倍关系，输出 BCLK 信号会产生抖动。

3.4 步骤 3：确定 CLB 逻辑所需的输出

如图 3-4 所示，输出 TDM-8 数据流信号通过 CLB OUTPUT XBAR 和 GPIO OUTPUT XBAR 连接到 CLB Tile（TILE）边界输出。

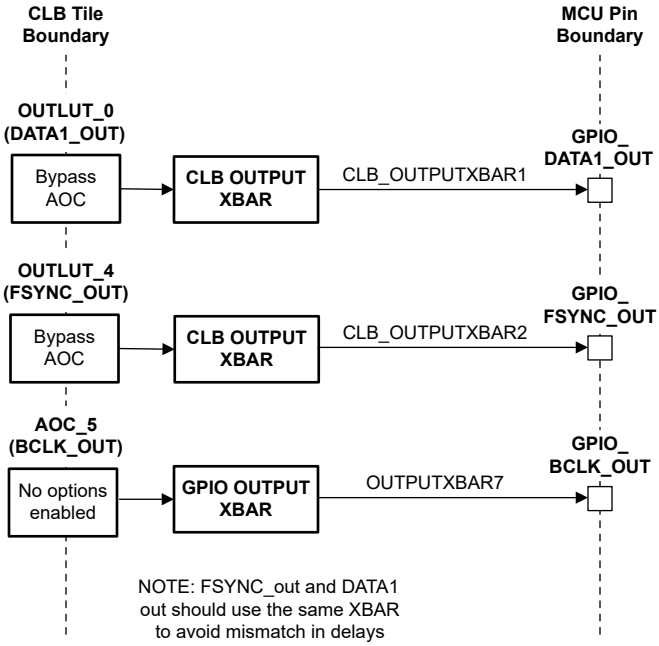


图 3-4. TDM-8 示例的 CLB Tile（TILE）输出

如前所述，输入 BCLK 信号必须以未寄存方式穿过 CLB Tile（TILE）（即不与任何内部时钟同步），以避免输出 BCLK 信号产生抖动。只有 CLB 输出 4 和 5 可以未寄存方式穿过 CLB Tile（TILE），因此使用 CLB 输出 5 作为 BCLK_OUT。此外，BCLK 信号通过 GPIO OUTPUT XBAR 引至 GPIO 引脚。

Copyright © 2023 Texas Instruments Incorporated

3.5 步骤 4：设计 CLB 逻辑

最终的 CLB Tile（TILE）逻辑设计如图 3-5 所示。

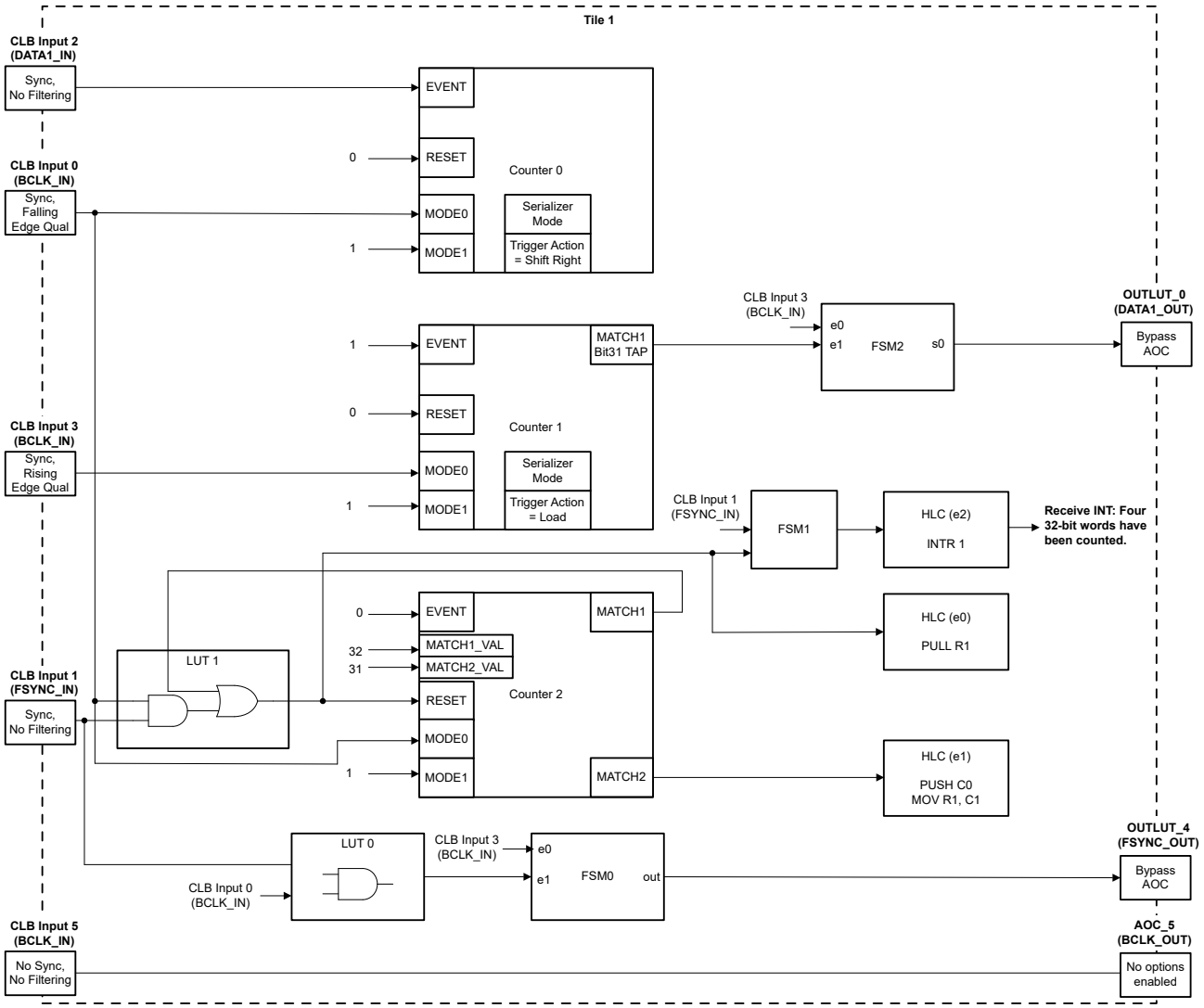


图 3-5. TDM-8 示例的 CLB Tile（TILE）逻辑

为简化 CLB 逻辑设计，采用图 3-6 所示的 TDM 时钟、帧同步和数据信号时序。可使用 MCU GPIO 引脚的反相功能，以支持相反的时钟和帧同步极性。

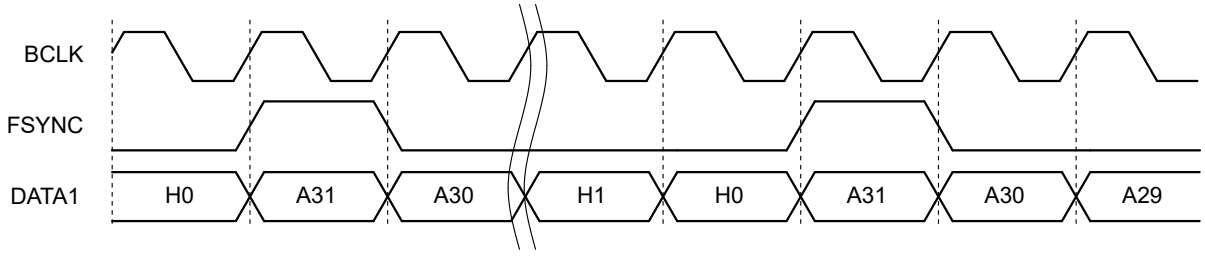


图 3-6. CLB 逻辑设计所采用的 TDM 数据流帧同步脉冲、时钟信号和数据信号时序

Copyright © 2023 Texas Instruments Incorporated

3.5.1 资源分配

CLB Tile（TILE）资源分配如下：

- COUNTER0 用于接收输入数据，并以串行器模式运行。在 BCLK_IN 下降沿，DATA_IN 移入计数器。
- COUNTER1 用于输出待发送数据，并以串行器模式运行。在 BCLK_IN 上升沿，数据移出 COUNTER1。
- COUNTER2 用于统计 BCLK_IN 的下降沿，并以普通计数器模式运行。当检测到 FSYNC + BCLK（低电平沿）条件，或位计数达到 32 时，COUNTER2 会复位。
- LUT1 用于在检测到 FSYNC_IN 和字边界时复位 COUNTER2。LUT1 将 FSYNC_IN 与 BCLK_IN 的下降沿进行逻辑 AND，以确保仅在确定 FSYNC_IN 有效时对其采样。LUT1 还会触发 HLC 从 PULL FIFO 读取新数据。
- 收到/发送 4 个 32 位字后，FSM1 会向 HLC 触发一个事件。检测到 FSYNC_IN 时，FSM1 会将计数复位。
- 检测到 FSYNC_IN 后，LUT0 和 FSM0 会生成 FSYNC_OUT 信号。FSYNC_OUT 与 BCLK_IN 上升沿对齐。请注意，FSYNC_OUT 并非 FSYNC_IN 的直通信号，因为本设计不要求直通。
- FSM2 用于使 DATA1_OUT 信号与 BCLK_IN 上升沿对齐。
- HLC 用于在 CLB push/pull FIFO 和两个串行器计数器（COUNTER0 和 COUNTER1）之间移动数据。每收到/发送 4 个 32 位字，HLC 还会向 CPU 产生一次中断。

3.5.2 TDM 字计数器

每收到/发送 4 个 32 位字，FSM1 就会触发一个 HLC 事件。FSM1 状态图和真值表分别见图 3-7 和表 3-1。

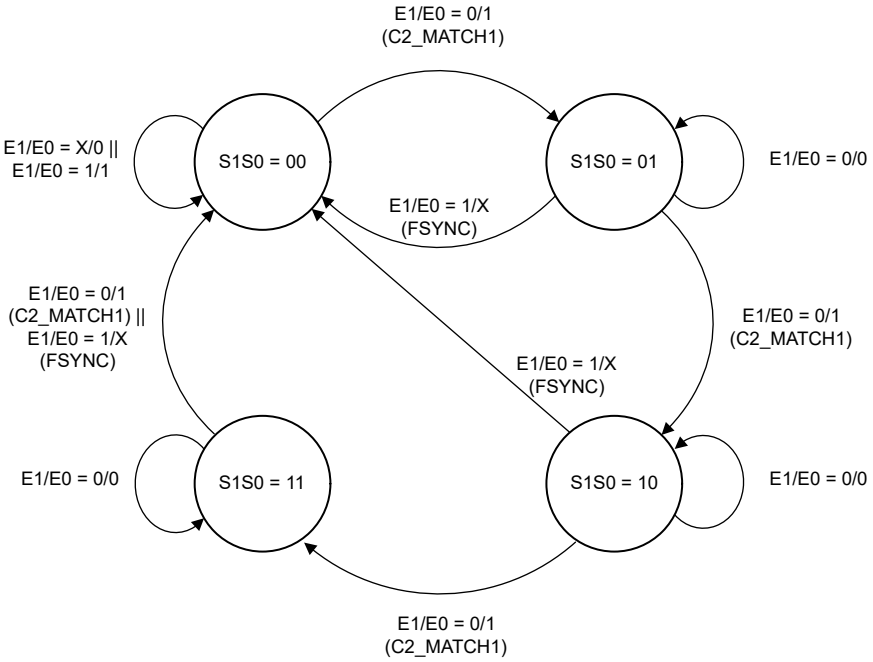


图 3-7. TDM-8 示例的 FSM1 状态图

Copyright © 2023 Texas Instruments Incorporated

表 3-1. TDM-8 示例的 FSM1 真值表

S1	S0	E1 (FSYNC)	E0 (C2_MATCH1)	S1 Next	S0 Next	Output (HLC_INT)
0	0	0	0	**0**	**0**	**0**
0	0	0	1	**0**	**1**	**0**
0	0	1	0	**0**	**0**	**0**
0	0	1	1	**0**	**0**	**0**
0	1	0	0	**0**	**1**	**0**
0	1	0	1	**1**	**0**	**0**
0	1	1	0	**0**	**0**	**0**
0	1	1	1	**0**	**0**	**0**
1	0	0	0	**1**	**0**	**0**
1	0	0	1	**1**	**1**	**0**
1	0	1	0	**0**	**0**	**0**
1	0	1	1	**0**	**0**	**0**
1	1	0	0	**1**	**1**	**0**
1	1	0	1	**0**	**0**	**1**
1	1	1	0	**0**	**0**	**1**
1	1	1	1	**0**	**0**	**1**

化简后的逻辑方程如下：

- S1 (NEXT) = S1&!S0&!E1 | S1&!E1&!E0 | !S1&S0&!E1&E0
- S0 (NEXT) = !S0&!E1&E0 | S0&!E1&!E0
- OUTPUT = S1&S0&E0 | S1&S0&E1

3.5.3 FSYNC 和 DATA1 输出同步

由于 CLB Tile（TILE）内部的数据移动延迟，DATA1_OUT 信号始终滞后于输入的 BCLK_IN 和 FSYNC_IN 信号（见图 3-10）。为使 FSYNC_OUT 和 DATA1_OUT 信号彼此对齐，使用两个 FSM 锁存并保持这两个信号。输出锁存器在 BCLK_IN 上升沿更新。

FSM0 用于检测并延迟 FSYNC_IN 信号，因此 FSYNC_IN 不会直接传递到 FSYNC_OUT。其状态图见图 3-8，相应的真值表见
表 3-2。

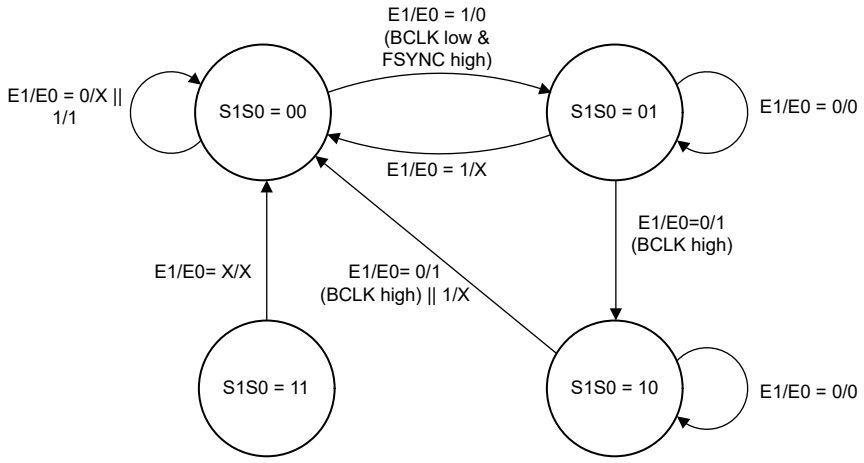


图 3-8. TDM-8 示例的 FSM0 状态机

Copyright © 2023 Texas Instruments Incorporated

表 3-2. TDM-8 示例的 FSM0 真值表

S1	S0	E1 (FSYNC high & BCLK low)	E0 (BCLK rise)	S1 Next	S0 Next	OUT
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	1	0
0	0	1	1	0	0	0
0	1	0	0	0	1	0
0	1	0	1	1	0	0
0	1	1	0	0	0	0
0	1	1	1	0	0	0
1	0	0	0	1	0	1
1	0	0	1	0	0	1
1	0	1	0	0	0	1
1	0	1	1	0	0	0
1	1	0	0	0	0	0
1	1	0	1	0	0	0
1	1	1	0	0	0	0
1	1	1	1	0	0	0

化简后的逻辑方程如下：

- S1 (NEXT) = !E1&!E0&S1&!S0 | !E1&E0&!S1&S0
- S0 (NEXT) = !E1&!E0&!S1&S0 | E1&!E0&!S1&!S0
- OUT = !E1&S1&!S0 | !E0&S1&!S0

FSM2 用于实现简单的 D 型触发器，以锁存并延迟 DATA1_OUT 信号。真值表见表 3-3。请注意，表 3-3 中的 S0 (next) 用于驱动最终的 DATA1_OUT 信号。

表 3-3. TDM-8 示例的 FSM2 真值表

E1 (DATA1)	E0 (BCLK rise)	S0	S0 (next)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

化简后的逻辑方程如下：

- S0 (NEXT) = !E0&S0 | E1&E0

Copyright © 2023 Texas Instruments Incorporated

3.6 步骤 5：仿真逻辑设计

图 3-9 展示了数据接收仿真。仿真使用简单的数据输入模式 0xAAAA AAAA，展示 TDM 帧中前一帧最后一个字的最后一位与新帧开始之间的转换。

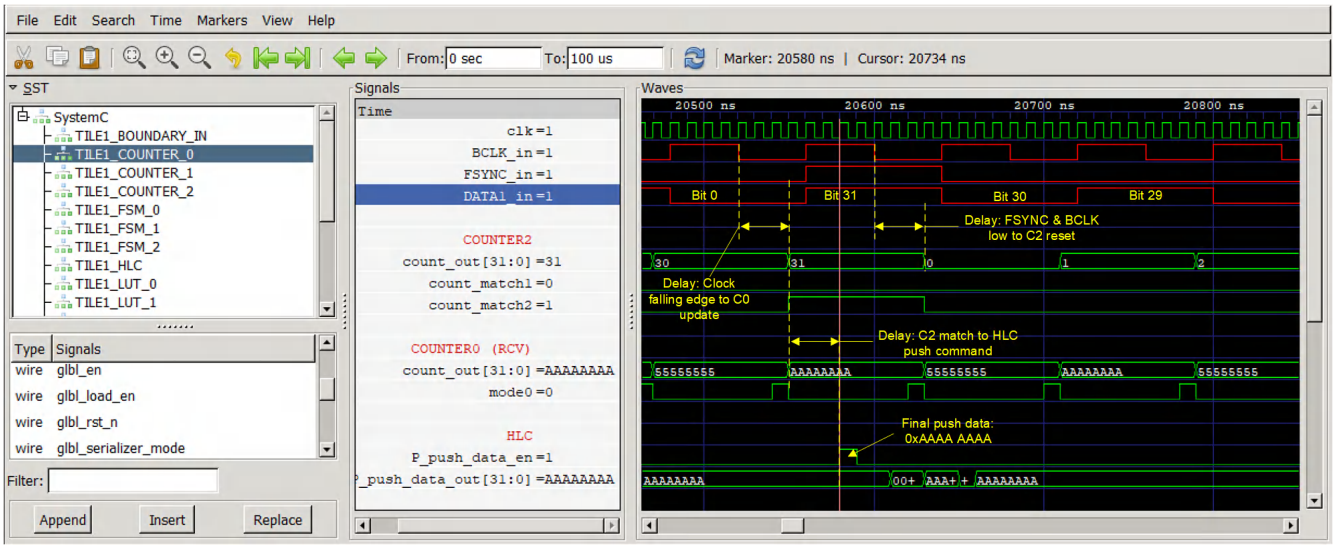


图 3-9. TDM-8 数据接收仿真

图 3-10 展示了数据发送仿真，其中显示 TDM 帧中前一帧最后一个字的最后一位与新帧开始之间的转换。仿真连续发送模式 0x5555 5554，以突出显示串行字之间串行器输出可能出现的短暂毛刺。为避免发送字的最高有效位丢失，串行器必须在计数值移位后 ***after*** 装载；因此，串行器输出会短暂无效。

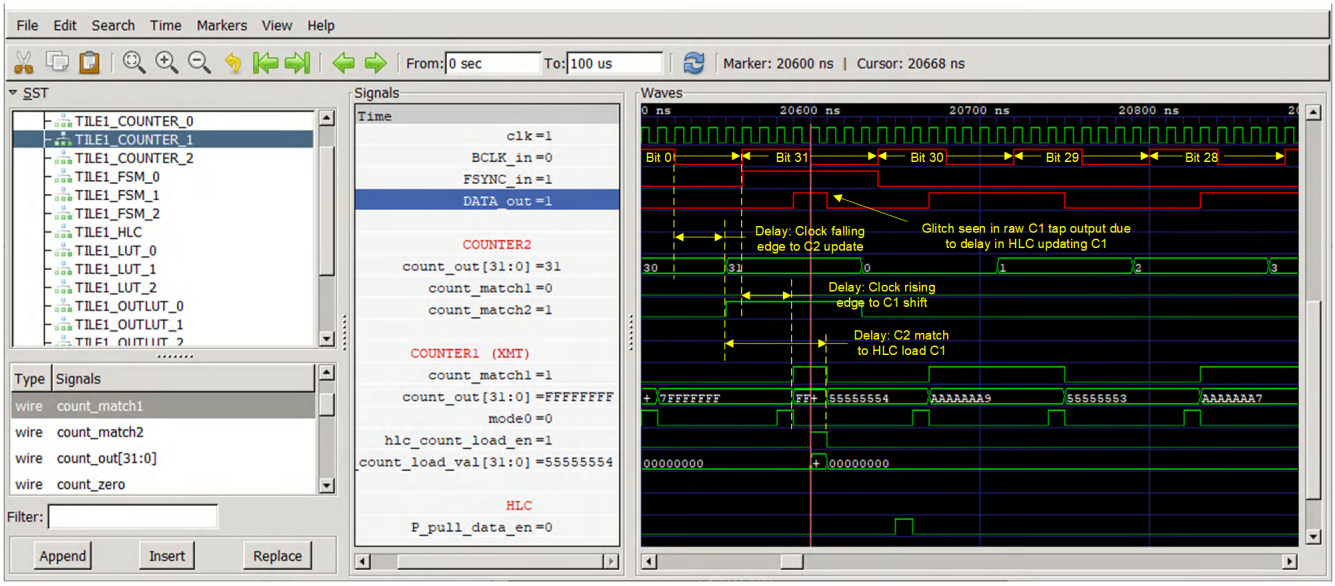


图 3-10. TDM-8 数据发送仿真

Copyright © 2023 Texas Instruments Incorporated

请注意，本 TDM 示例将 HLC 配置为先将 C0 中的值推送到 FIFO，再向 C1 装载新值（见图 3-5），以确保 C1 在 BCLK_IN 上升沿 *after* 装载。由于 HLC 操作由 BCLK 下降沿触发，而串行器操作取决于 BCLK 上升沿，因此系统依赖于 BCLK_IN 周期。如果 BCLK_IN 周期延长（即 BCLK_IN 频率降低或 CLB 时钟频率提高），C1 更新可能会在时钟上升沿之前发生，从而导致串行字的最高有效位丢失。

如图 3-10 所示，输出数据相对于输入 FSYNC 和 BCLK 有延迟。要使输出 FSYNC 和 DATA1 信号彼此同步，使用两个 FSM 锁存并延迟这两个信号。图 3-11 展示了此功能的仿真结果。

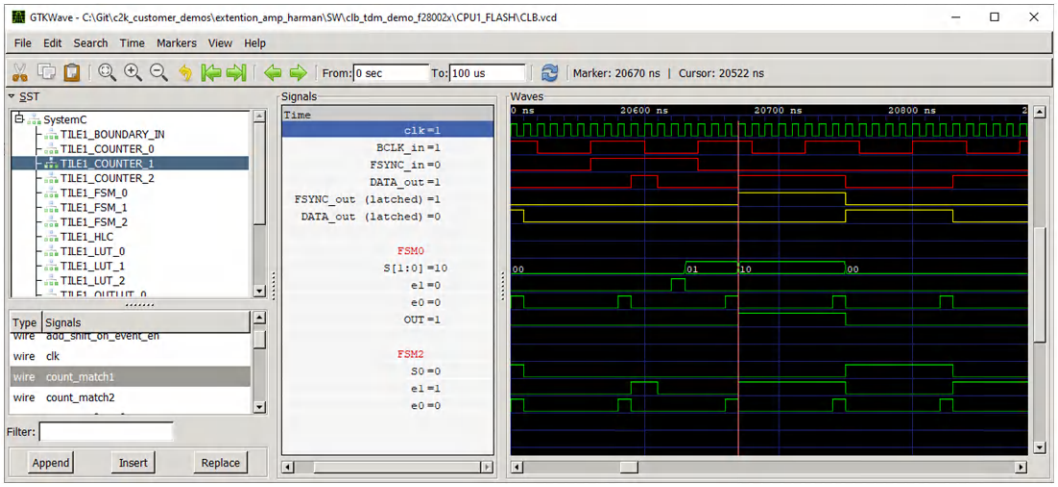


图 3-11. FSYNC 和 DATA1 同步

3.7 步骤 6：测试 CLB 逻辑

本节介绍如何测试随 CLB TDM-8 示例提供的 Code Composer Studio™ 项目。该示例已在 F280025C ControlCARD（TMDSCNCD280025C）上进行测试，也可轻松移植到支持 CLB Type 2 及以上版本的其他 MCU。

测试中使用另一块 F28388D MCU，通过 McBSP 端口生成测试用 TDM-8 数据流。F28388D 还会接收来自 CLB 的 TDM-8 数据流并检查错误。本测试使用 F28388D ControlCARD（TMDSCNCD28388D）。

本测试中，F28388D 配置为以 200MHz 运行，McBSP 生成 12.5MHz 的 TDM 波形。F280025C 配置为以最高 100MHz 运行。

图 3-12 显示了测试 TDM-8 示例所需的连接设置。

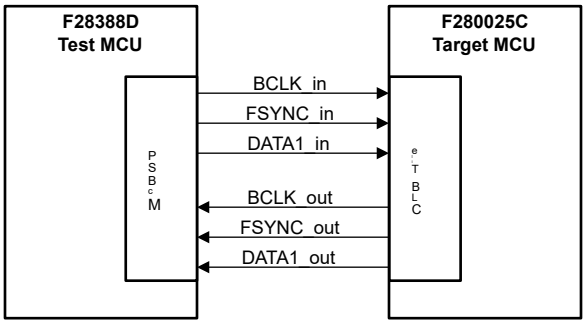


图 3-12. TDM 示例测试设置

Copyright © 2023 Texas Instruments Incorporated

3.7.1 硬件设置和连接

运行演示所需的硬件如下：

- TMDSCNCD280025C ControlCARD + TMDSHSECDOCK 底板扩展坞 + 5 V 电源
- TMDSCNCD28388D ControlCARD + TMDSHSECDOCK 底板扩展坞 + 5 V 电源

可选设备

- 逻辑分析仪（用于查看 TDM 总线信号）

按以下步骤连接硬件：

1. 将两块 ControlCARD 分别插入对应的 TMDSHSECDOCK。两块 ControlCARD 均应使用默认设置（参阅各 EVM 对应的用户指南）。2. 按表 3-4 和表 3-5 所示，将 F28388D ***output*** TDM 数据流连接到 F280025C ***input*** TDM。

表 3-4. F280025C CLB 逻辑输入 TDM 信号

TDM Input Pin	GPIO (BALL)	DOCK Pin
FSYNC_IN	GPIO00	49
BCLK_IN	GPIO01	51
DATA1_IN	GPIO02	53

表 3-5. F28388D McBSP 输出 TDM 信号

TDM Input Pin	GPIO (BALL)	DOCK Pin
MCLKX/BCLK	GPIO22	72
MFSX/FSYNC	GPIO23	74
MDX/DATA1	GPIO20	68

3. 按表 3-6 和表 3-7 所示，将 F280025C ***output*** TDM 数据流连接到 F28388D ***input*** TDM。

表 3-6. F280025C CLB 逻辑输出 TDM 信号

TDM Output Pin	GPIO (BALL)	DOCK Pin
FSYNC_OUT	GPIO04	50
BCLK_OUT	GPIO05	52
DATA1_OUT	GPIO06	54

表 3-7. F28388D McBSP 输入 TDM 信号

TDM Input Pin	GPIO (BALL)	DOCK Pin
MCLKR/BCLK	GPIO58	108
MFSR/FSYNC	GPIO59	110
MDR/DATA1	GPIO21	70

4. 在两个扩展坞之间连接至少两处公共 GND。5. 将相应的 USB 线缆连接到每块 ControlCARD。6. 将 5V 电源连接到每个 TMDSHSECDOCK（也可另用 USB 线缆为扩展坞供电）。7. 将每个 TMDSHSECDOCK 上的 S1 开关拨至“EXT_ON”位置。

Copyright © 2023 Texas Instruments Incorporated

3.7.2 软件设置

C2000Ware 中提供了两个 CCS 项目，可用于测试此 CLB 示例。TDM 项目的路径如下：

- <C2000WARE_INSTALL>/driverlib/f28002x/examples/clb
- <C2000WARE_INSTALL>/driverlib/f2838x/examples/c28x/mcbsp

构建和运行软件需要以下软件包：

- Code Composer Studio（CCS）11.1.00 版或更高版本
- C2000Ware 4.3.00.00 版或更高版本
- GNU 编译器（TDM-GCC）和 GTK Wave 仿真查看器（运行 CLB 仿真时可选）。更多信息请参阅 *CLB Tool's User Guide*。

首先，按照以下步骤在 CCS 中设置 F280025C：

1. 在 CCS 菜单中单击“Project -> Import CCS Projects...”。2. 在“Select search-directory”中输入 CLB 示例项目路径。a. 路径：<C2000WARE_INSTALL>/driverlib/f28002x/examples/clb 3. 单击“Refresh”。4. 选择“clb_ex31_tdm_serial_port”项目。

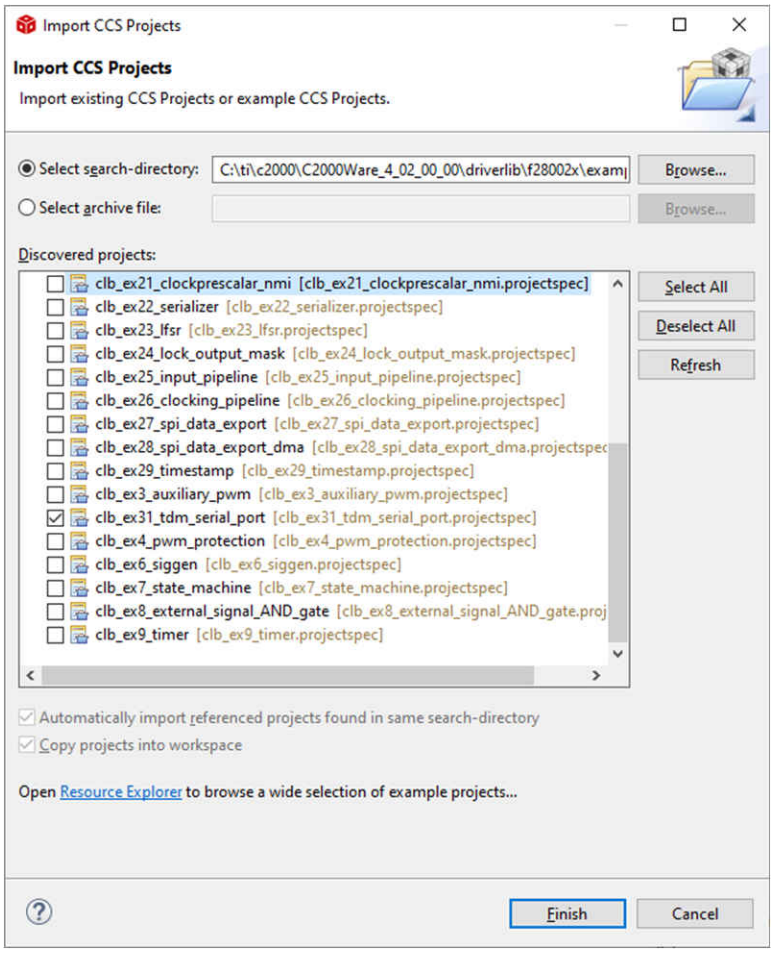


图 3-13. 导入 CLB TDM 示例项目

5. 勾选“Copy projects into workspace”。6. 单击“Finish”，将项目导入工作区。

可选：单击“Project -> Build Configurations -> Set Active -> CPU1_FLASH”，以构建用于 Flash 运行的代码。将 TMDSCNCD280025C ControlCARD 设置为从 Flash 引导（参阅 *TMS320F28388D controlCARD Information Guide*）。7. 在 CCS 菜单中单击“Project -> Build Project”，构建示例项目。8. 单击“Run -> Debug”，将可执行文件加载到 F280025C 目标器件。

Copyright © 2023 Texas Instruments Incorporated

9. 最后，在 CCS 的 Debug 透视图中单击“Run -> Resume”以运行代码。

其次，按照以下步骤在 CCS 中设置 F28388D，以生成测试用 TDM 数据流：

1. 在 CCS 菜单中单击“Project -> Import CCS Projects...”。2. 在“Select search-directory”中输入 McBSP 示例项目路径。a. 路径：<C2000WARE_INSTALL>/driverlib/f2838x/examples/c28x/mcbsp 3. 单击“Refresh”。4. 选择“mcbsp_ex7_tdm8_test”项目。

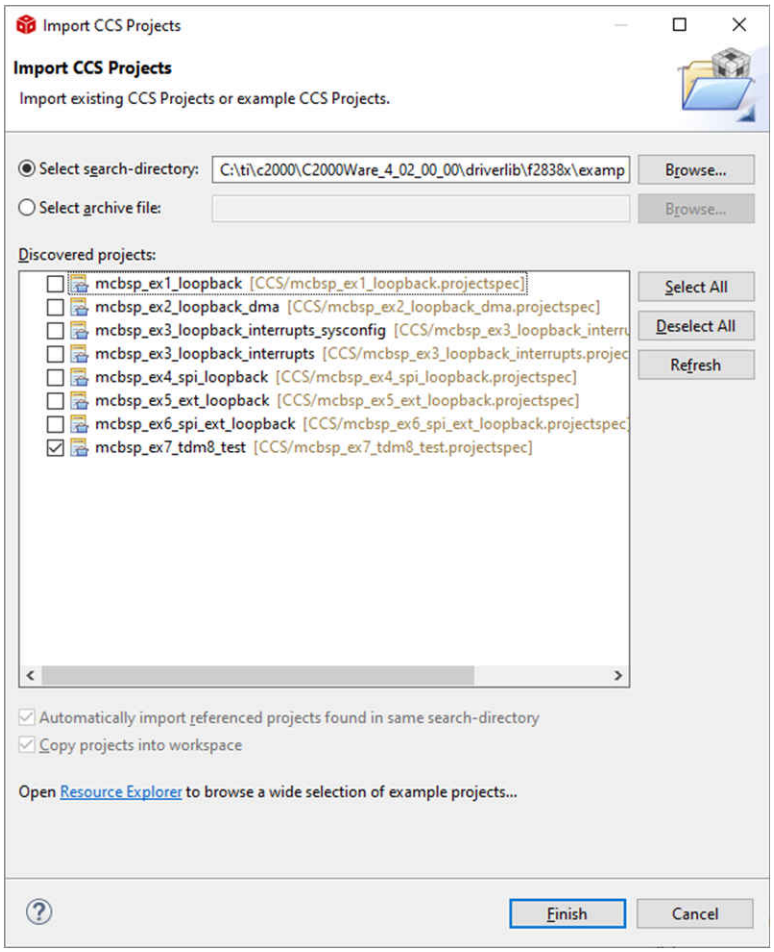


图 3-14. 导入 McBSP TDM 示例项目

5. 勾选“Copy projects into workspace”。6. 单击“Finish”，将项目导入工作区。a. 可选：单击“Project -> Build Configurations -> Set Active -> CPU1_FLASH”，以构建用于 Flash 运行的代码。将 TMDSCNCD28388D ControlCARD 设置为从 Flash 引导（参阅 ControlCARD 用户指南）。7. 在 CCS 菜单中单击“Project -> Build Project”，构建示例项目。8. 单击“Run -> Debug”，将可执行文件加载到 F28388D 目标器件。a. 可选：将以下全局变量添加到 Expressions 窗口：txData、rxData、testWordDetected 和 errCountGlobal。9. 最后，在 CCS 的 Debug 透视图中单击“Run -> Resume”以运行代码。

3.7.3 测试输出建立时间和保持时间

CLB 输出 TDM 数据流的一个关键考量是接收器件所见的建立时间和保持时间。为测量这两个时间，使用示波器持续采集 CLB 输出。图 3-15 显示了未为 FSYNC_OUT 和 DATA1_OUT 信号添加锁存和延迟逻辑时的 TDM 输出。由于 DATA1_OUT 信号存在延迟，其相对于 BCLK_OUT 的建立时间和保持时间缩短。

Copyright © 2023 Texas Instruments Incorporated

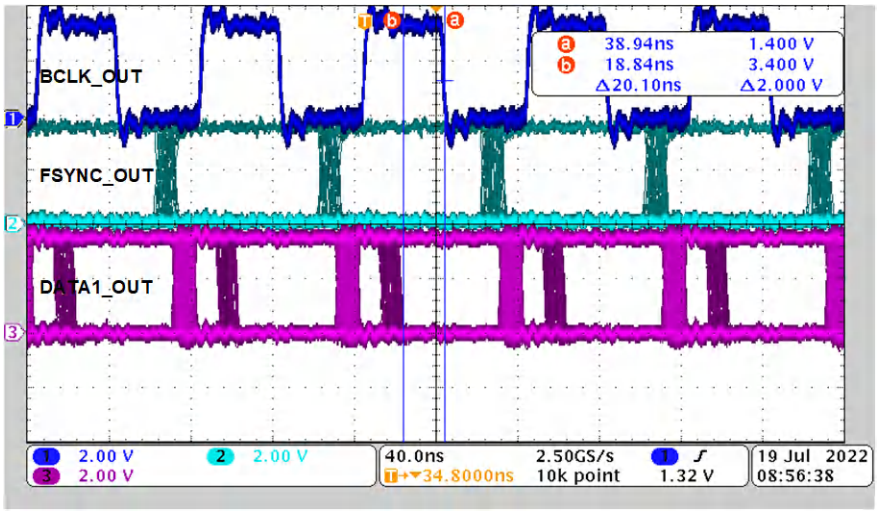


图 3-15. 未添加锁存和延迟逻辑时的 TDM-8 输出

图 3-16 的示波器波形显示了为 FSYNC_OUT 和 DATA1_OUT 信号添加锁存和延迟逻辑后的最终 TDM 输出。采用此方法后，建立时间和保持时间均显著改善。

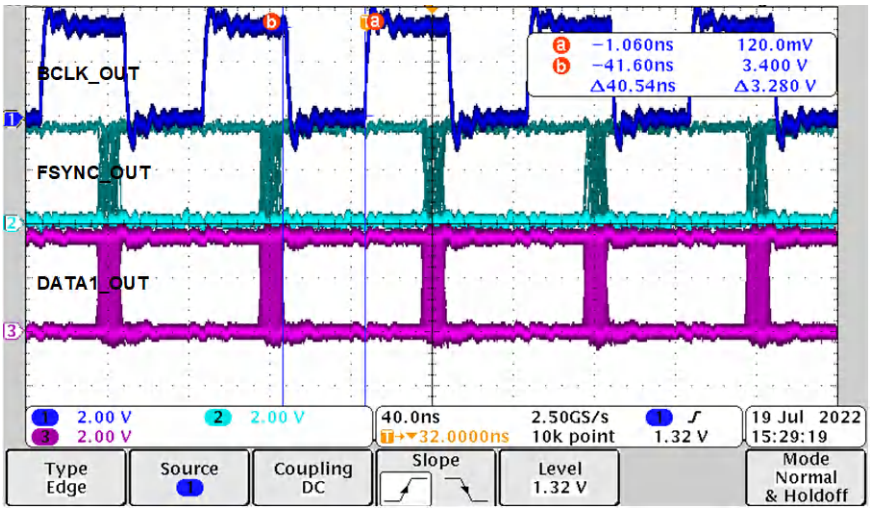


图 3-16. 最终 TDM-8 输出

由于 BCLK_OUT 必须无抖动地直通 CLB Tile（TILE），因此在不使 BCLK_OUT 产生抖动的情况下，无法精确控制其与另外两个信号之间的时序。不过，本例中的建立时间和保持时间均可接受。如有需要，可使用 CLB Tile（TILE）中的 AOC 反相 BCLK_OUT 信号，以建立时间换取保持时间。

最后，请注意 FSYNC_OUT 和 DATA1_OUT 信号存在 10 ns 抖动。这是因为 CLB Tile（TILE）以 100 MHz 对输入 BCLK_IN 信号进行采样。抖动会进一步缩短建立时间和保持时间。

Copyright © 2023 Texas Instruments Incorporated

3.7.4 测试数据完整性

为测试数据完整性，F28388D 向 F280025C 发送递增的数据模式。F280025C 将接收数据与预期数据进行比较，并在检测到错误时置位标志。F280025C 会将接收到的所有数据回传给 F28388D，最后由 F28388D 检查所有接收数据是否有误。图 3-17 显示了 F28388D 正在监测输入数据的情况。整个 32 位范围的数据传输过程中，F280025C 和 F28388D 均未检测到错误。

Variables | Expressions | Registers

Expression	Type	Value	Address
txData	unsigned long	0x843B2B6A (Hex)	0x0000A804@Data
testWordDetected	unsigned int	1	0x0000A800@Data
errCountGlobal	unsigned long	0	0x0000A802@Data
Add new expression			

图 3-17. F28388D 监测输入的 TDM 流量

4 示例 B：在照明应用中使用 CLB 为 LED 驱动器实现自定义通信总线

4.1 示例概述

本示例使用 C2000 实时控制微控制器，在 16 x 16 RGB LED 点阵显示屏上显示简单图案。该显示屏由一个 LP5891-Q1 LED 驱动器驱动，并通过串行总线与 C2000 实时控制微控制器通信。

本示例使用两个 CLB Tile（TILE）支持 LP5891-Q1 LED 驱动器所需的连续时钟串行接口（CCSI）总线协议。基于 CLB 实现的 CCSI 总线只需极少的 CPU 开销即可对总线帧编码，因为 CLB 会自动添加 CCSI 通信所需的 IDLE、START、END 和 CHECK 位。

可修改示例代码，以级联另一块 LED 点阵显示屏。

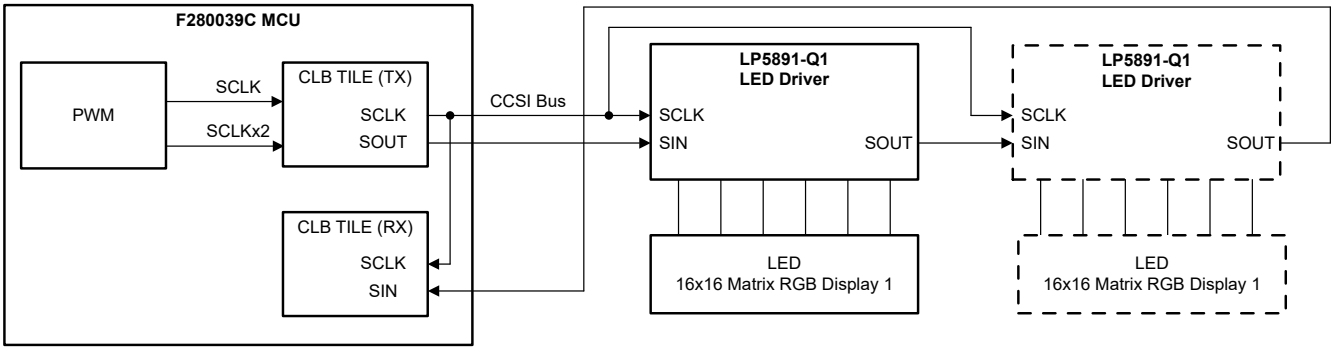


图 4-1. LED 点阵示例框图

Copyright © 2023 Texas Instruments Incorporated

4.2 步骤 1：了解设计要求

本演示采用的设计参数列于表 4-1。

表 4-1. 设计要求

Design Parameter	Value
Display module size	16 x 16 RGB LEDs
Frame rate	60 Hz
Refresh rate	7680 Hz
PWM resolution	16 bits
Cascaded devices	1 (can be increased to 2)
CCSI bus count	1
SCLK frequency	5.0 MHz
GCLK frequency	70.0 MHz
T_{SW}	643 ns

图 4-2 显示了 LP5891-Q1 LED 驱动器支持的 CCSI 总线协议。该协议要求如下：

- 连续串行时钟
- 可变帧长度，包含起始、空闲、数据和结束状态
- 16 位头字和数据字
- 所有头字和数据字均附加校验位

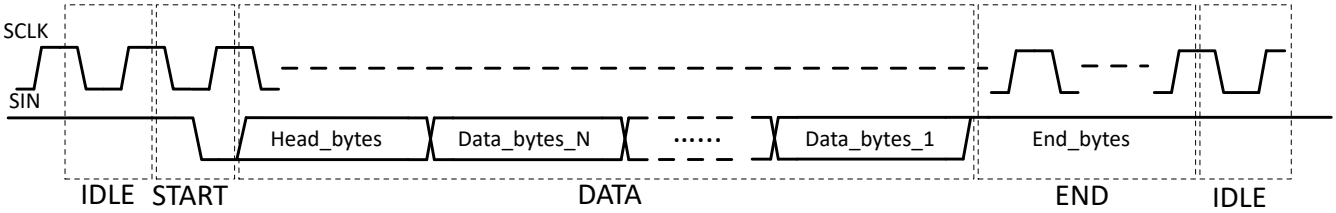


图 4-2. CCSI 帧

此外，还有两项要求。第一，为支持不同的 LED 驱动器件，需要能够配置单时钟边沿或双时钟边沿的数据收发。例如，TLC6983 LED 显示驱动器使用双时钟边沿收发数据。第二，还需能够在多个 CCSI 总线上同时生成 VSYNC 命令。VSYNC 用于同步级联链中各器件的每帧显示。

Copyright © 2023 Texas Instruments Incorporated

4.3 步骤 2：确定 CLB Tile（TILE）所需的输入

TX Tile 的输入信号如图 4-3 所示。

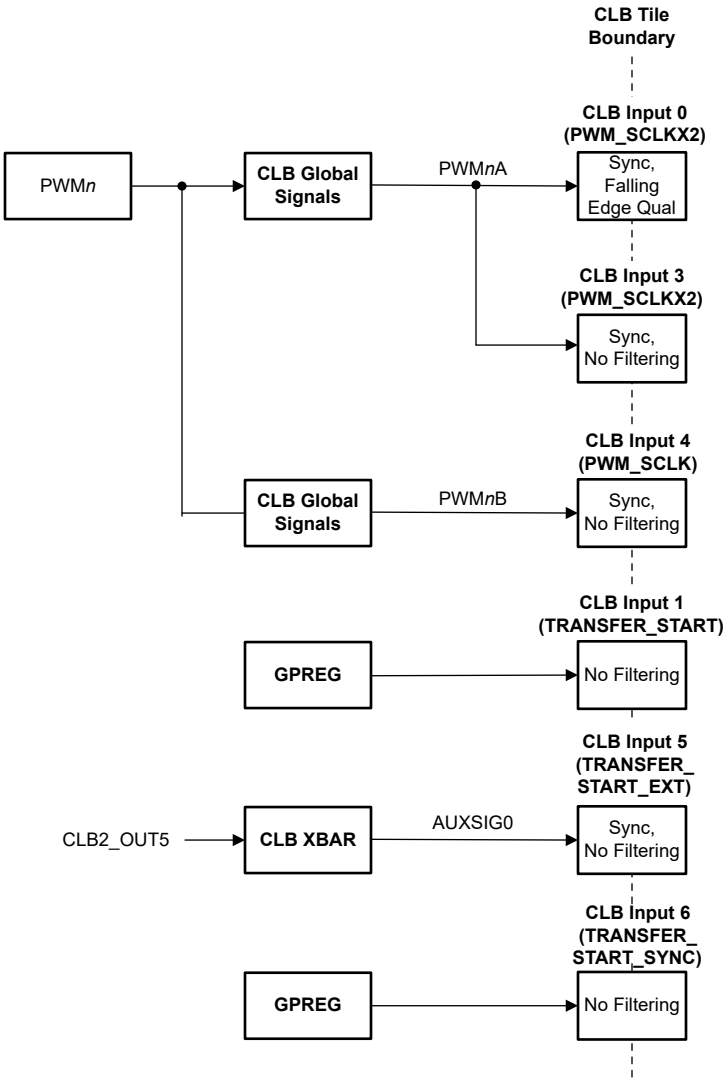


图 4-3. TX Tile 输入信号

TX Tile 使用以下输入：

- CLB Input 0：该输入由内部 PWM*n*A 输出驱动。PWM*n*A 输出配置为生成频率是目标 SCLK 2 倍的时钟。TX Tile 使用 PWM*n*A 时钟发送数据。
- CLB Input 3：该输入由内部 PWM*n*A 输出驱动。该输入穿过 Tile 后用于驱动 CLB_SCLKX2 输出。
- CLB Input 4：该输入由内部 PWM*n*B 输出驱动。PWM*n*B 输出配置为生成目标 SCLK 频率的时钟。该输入穿过 Tile 后用于驱动 CLB_SCLK 输出。

TX Tile 还使用 3 个辅助信号，以同步所有 TX Tile 上的数据发送：

- CLB Input 1：GPREG.1 位用于启动 TX Tile 上的数据传输，由 CPU 置位/清零。
- CLB Input 5：此输入用作 TX Tile 的外部传输启动信号，始终取自 CLB Tile 2 的输出 5。
- CLB Input 6：当使用多个 TX Tile 时，GPREG.6 位用作所有 TX Tile 的外部传输启动信号。只有 CLB Tile 2 中的 GPREG.6 位可用于启动其他 TX Tile 上的传输。GPREG.6 位由 CPU 置位/清零。

Copyright © 2023 Texas Instruments Incorporated

RX Tile 的输入信号如图 4-4 所示。

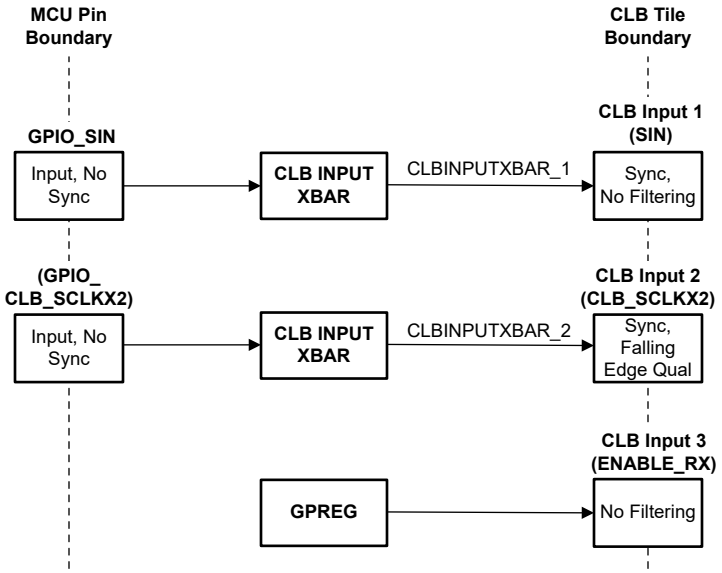


图 4-4. RX Tile 输入信号

RX Tile 使用以下输入：

- CLB Input 1：此输入用于从 SIN 引脚接收数据。
- CLB Input 2：此输入用于在 CLB_SCLKX2 下降沿触发数据接收。CLB_SCLKX2 信号是由 TX CLB Tile 提供的外部时钟，其频率为 SCLK 的 2 倍。
- CLB Input 3：GPREG.3 位用于启用/禁用数据接收，由 CPU 置位/清零。

4.4 步骤 3：确定 CLB 逻辑所需的输出

TX Tile 的输出信号如图 4-5 所示。

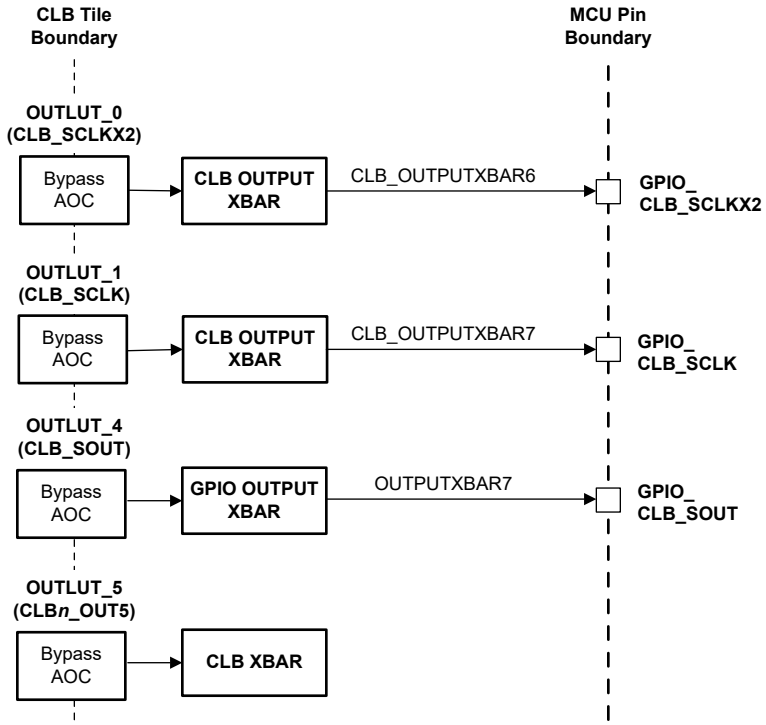


图 4-5. TX Tile 输出信号

Copyright © 2023 Texas Instruments Incorporated

PWM 在内部生成的 SCLK 和 SCLKX2 时钟会穿过 TX Tile，并由 GPIO 引脚输出。TX Tile 的输出由 CLB_SOUT 引脚驱动。最后，Tile 的输出 5 被送至 CLB XBAR，可用于同步多个 Tile 上的数据发送。

RX Tile 逻辑没有输出信号。

4.5 步骤 4：设计 CLB 逻辑

最终设计使用两个 CLB Tile 实现一个 CCSI 串行端口：一个 Tile 用于发送，另一个 Tile 用于接收。可将 TX 和 RX Tile 逻辑复制到多个 Tile，以实现更多 CCSI 总线。

4.5.1 TX Tile 逻辑

TX Tile 的逻辑如图 4-6 所示。

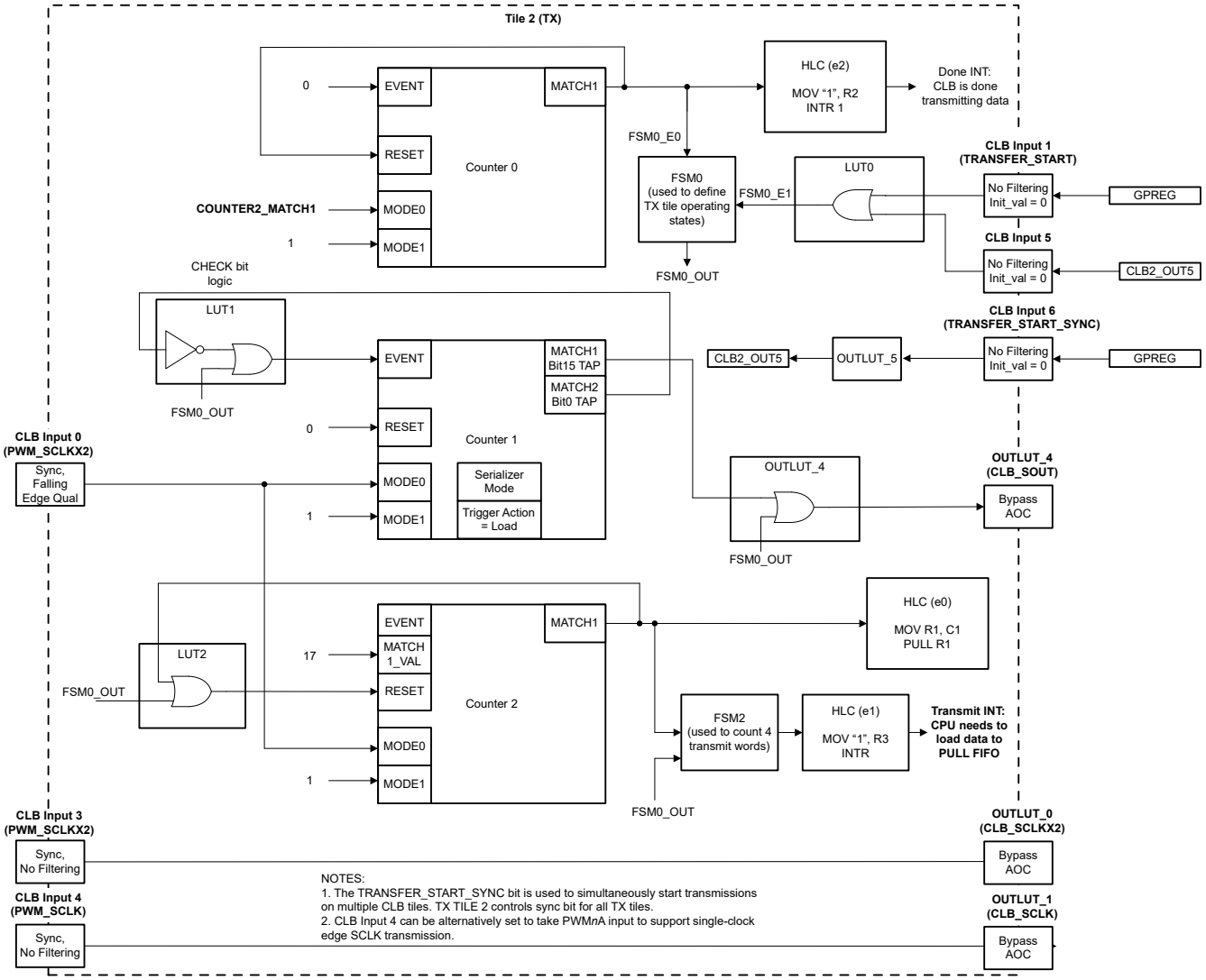


图 4-6. CLB TX Tile 逻辑

以下 CLB Tile 资源用于实现 CCSI 总线发送（TX）功能：

- COUNTER0 用于统计待发送的数据字数，并以普通计数器模式运行。
- COUNTER1 用于在 SCLKX2 下降沿发送输出数据，并以串行器模式运行。
- COUNTER2 用于统计 SCLKX2 下降沿的数量，并以普通计数器模式运行。
- HLC 用于从 CLB PULL FIFO 向 COUNTER1 传送数据。传输 4 个 16 位字或完整帧后，HLC 会向 CPU 产生中断。
- FSM0 用于在 IDLE、ACTIVE 和 END 状态之间循环切换。

Copyright © 2023 Texas Instruments Incorporated

- 发送 4 个 16 位字时，FSM2 负责计数。发送计数达到 4 后，会向 HLC 产生一个事件。

多个 LUT 和输出 LUT 用于实现组合逻辑。

FSM0 为 TX Tile 定义了 3 种运行状态。FSM0 状态图和真值表分别见图 4-7 和表 4-2。

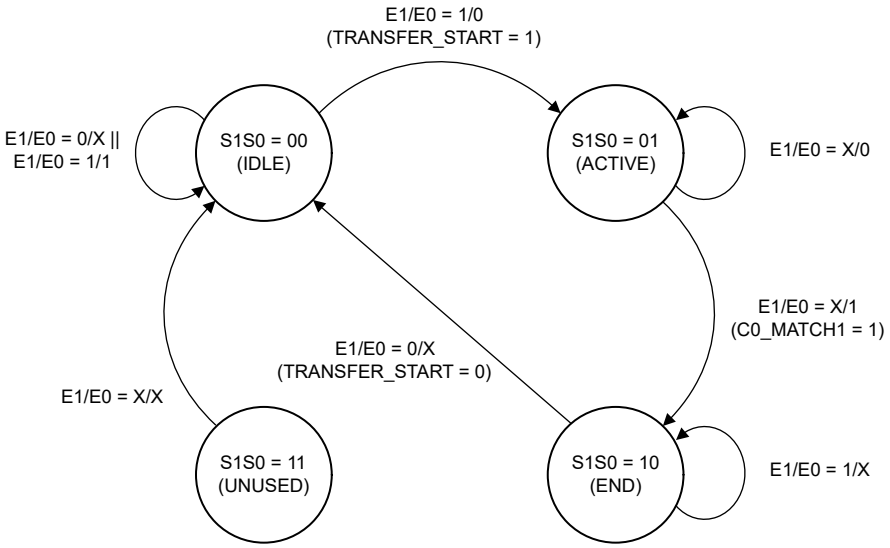


图 4-7. TX Tile 的 FSM0 状态图

表 4-2. FSM0 真值表

S1	S0	E1 (TRANSFER_ START)	E0 (C0_MATCH1)	S1 Next	S0 Next	Output (FSM0_OUT)
0	0	0	0	0	0	1
0	0	0	1	0	0	1
0	0	1	0	0	1	1
0	0	1	1	0	0	1
0	1	0	0	0	1	0
0	1	0	1	1	0	0
0	1	1	0	0	1	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	1	0	1
1	0	1	1	1	0	1
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	0	1

化简后的逻辑方程如下：

- S1 = !S1&S0&E0 | S1&!S0&E1
- S0 = !S1&E1&!E0 | !S1&S0&!E0
- OUTPUT = !S0 | S1

发送 4 个 16 位字后，FSM2 会触发 HLC 事件。FSM2 状态图和真值表分别见图 4-8 和表 4-3。

Copyright © 2023 Texas Instruments Incorporated

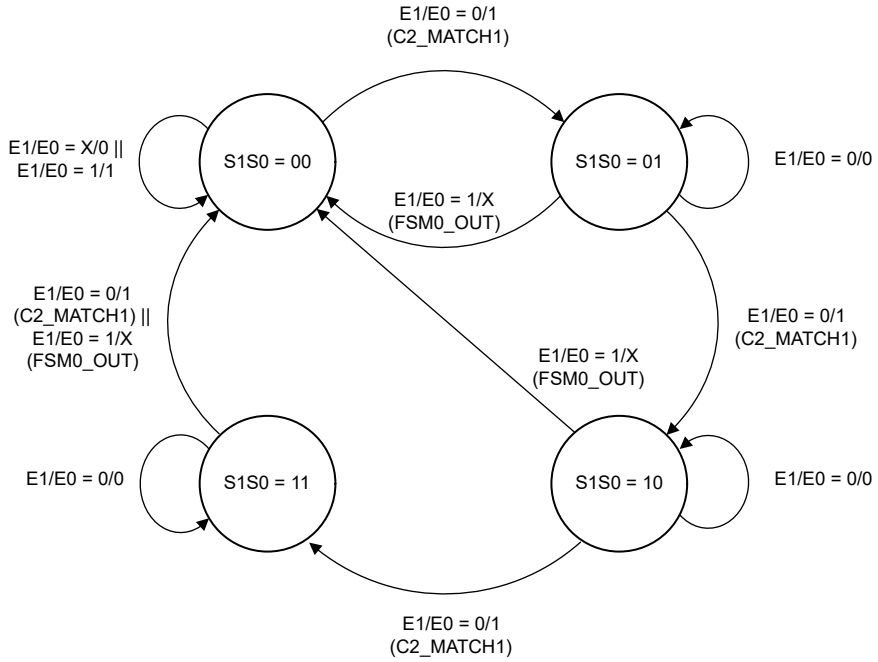


图 4-8. TX Tile 的 FSM2 状态图

表 4-3. FSM2 真值表

S1	S0	E1 (FSM0_OUT)	E0 (C2_MATCH1)	S1 Next	S0 Next	Output (HLC_INT)
0	0	0	0	**0**	**0**	**0**
0	0	0	1	**0**	**1**	**0**
0	0	1	0	**0**	**0**	**0**
0	0	1	1	**0**	**0**	**0**
0	1	0	0	**0**	**1**	**0**
0	1	0	1	**1**	**0**	**0**
0	1	1	0	**0**	**0**	**0**
0	1	1	1	**0**	**0**	**0**
1	0	0	0	**1**	**0**	**0**
1	0	0	1	**1**	**1**	**0**
1	0	1	0	**0**	**0**	**0**
1	0	1	1	**0**	**0**	**0**
1	1	0	0	**1**	**1**	**0**
1	1	0	1	**0**	**0**	**1**
1	1	1	0	**0**	**0**	**0**
1	1	1	1	**0**	**0**	**0**

化简后的逻辑方程如下:

- S1 = S1&!S0&!E1 | S1&!E1&!E0 | !S1&S0&!E1&E0
- S0 = !S0&!E1&E0 | S0&!E1&!E0
- OUTPUT = S1&S0&!E1&E0

Copyright © 2023 Texas Instruments Incorporated

4.5.2 RX Tile 逻辑

接收 Tile 的逻辑如图 4-9 所示。

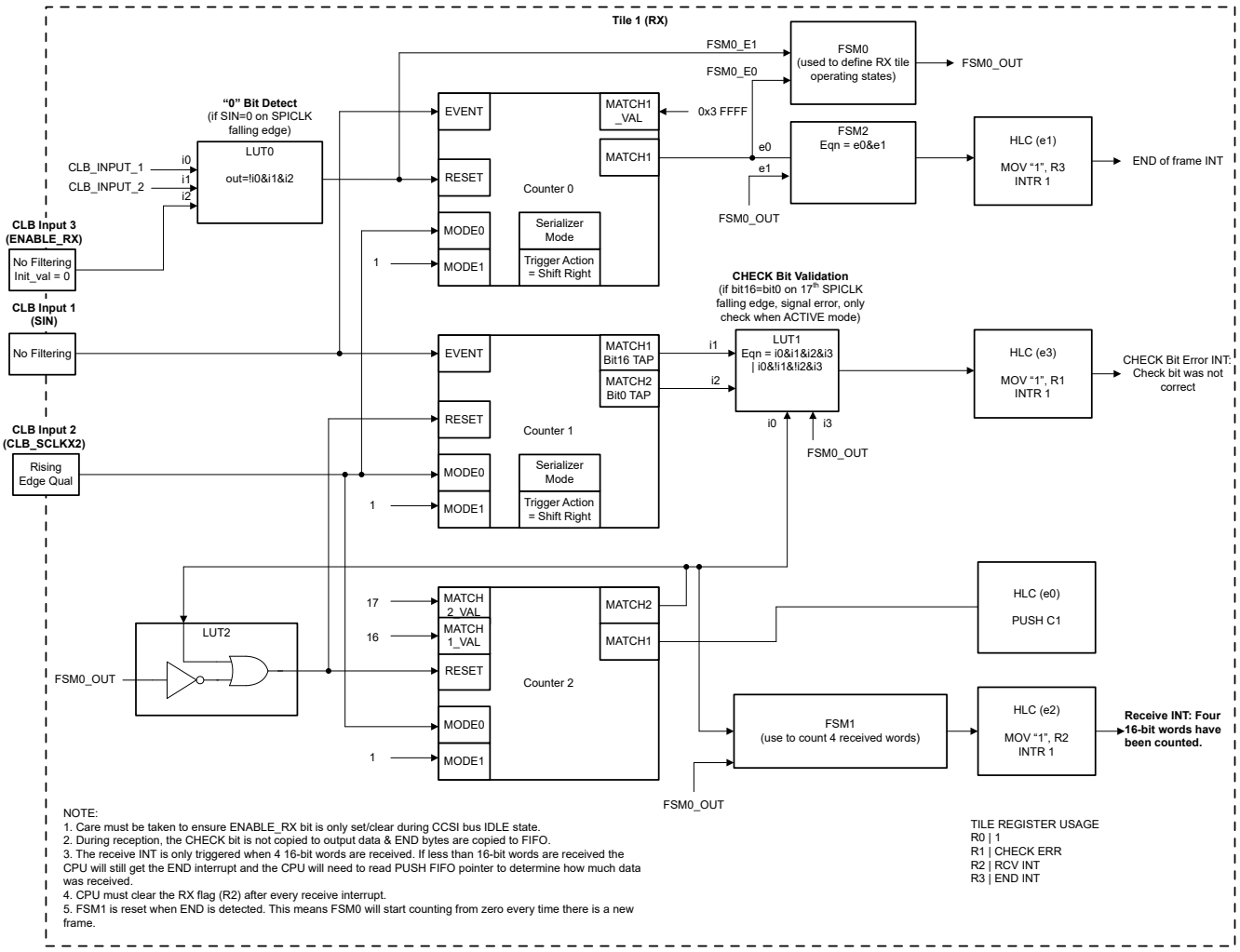


图 4-9. CLB RX Tile 逻辑

以下 CLB Tile 资源用于实现 CCSI 总线接收（RX）功能：

- COUNTER0 用于检测接收帧的结束。
- COUNTER1 用于在 SCLKX2 下降沿接收输入数据，并以串行器模式运行。
- COUNTER2 用于统计 SCLKX2 下降沿的数量，并以普通计数器模式运行。
- HLC 用于将数据从 COUNTER1 传送到 CLB PUSH FIFO。收到 4 个 16 位字、检测到 CHECK 位错误或检测到帧结束条件时，HLC 还会向 CPU 产生中断。

- FSM0 用于定义并循环切换两个状态：IDLE 和 ACTIVE。
- 每收到 4 个 16 位字，FSM1 就会计数。接收计数达到 4 后，会向 HLC 产生一个事件。

- FSM2 仅在 ACTIVE 状态下产生帧结束中断。
- LUT0 用于检测相应的“0”位，以识别 START 位和 END 字节。清除 ENABLE_RX 输入可将 LUT0 输出强制拉低，从而禁用接收操作。

- LUT1 用于检查每个 16 位字中的 CHECK 位。若 CHECK 位与预期状态不匹配，LUT1 输出会触发 HLC 中断。
- 禁用接收操作时（即 ENABLE_RX = 0），LUT2 用于使 COUNTER2 保持复位状态。

Copyright © 2023 Texas Instruments Incorporated

FSM0 为 RX Tile 定义了两种运行状态。FSM0 状态图和真值表见图 4-10 和表 4-4。

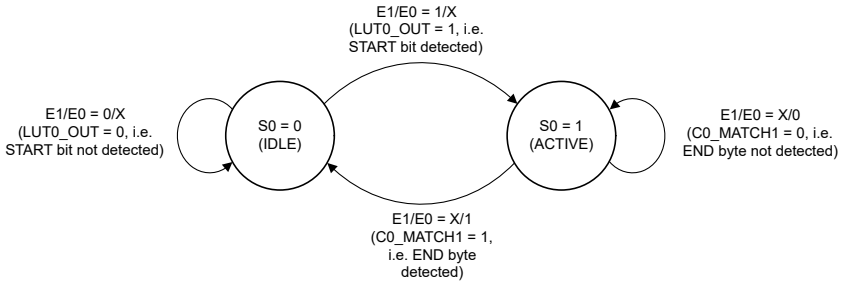


图 4-10. RX Tile 的 FSM0 状态图

表 4-4. FSM0 真值表

S0	E1 (LUT0_OUT)	E0 (C0_MATCH1)	S0 Next	Output (FSM0_OUTPUT)
0	0	0	0	0
0	0	1	0	0
0	1	0	1	0
0	1	1	1	0
1	0	0	1	1
1	0	1	0	1
1	1	0	1	1
1	1	1	0	1

完整逻辑方程如下：

- S0 = !S1&!S0&E1&!E0 | !S1&!S0&E1&E0 | !S1&S0&!E1&!E0 | !S1&S0&E1&!E0
- OUTPUT = !S1&S0&!E1&!E0 | !S1&S0&!E1&E0 | !S1&S0&E1&!E0 | !S1&S0&E1&E0

FSM1 用于每收到 4 个 16 位字后触发一个 HLC 事件。FSM1 状态图和真值表见图 4-11 和表 4-5。

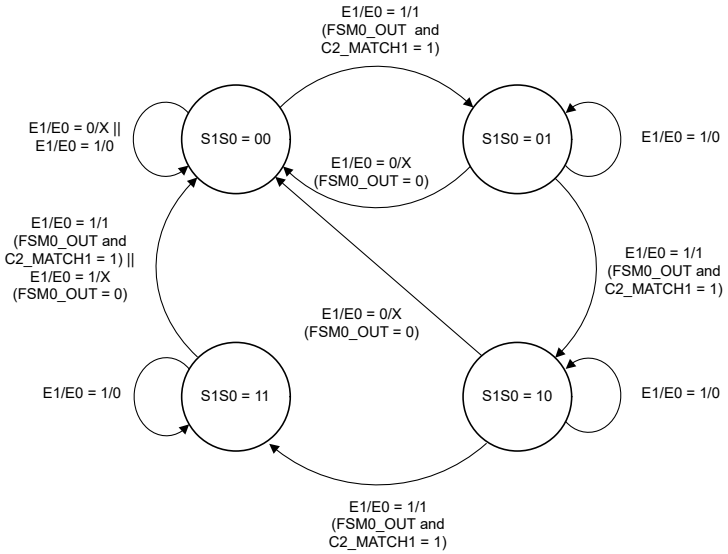


图 4-11. RX Tile 的 FSM1 状态图

Copyright © 2023 Texas Instruments Incorporated

表 4-5. FSM1 真值表

S1	S0	E1 (FSM0_OUT)	E0 (C2_MATCH2)	S1 Next	S0 Next	Output (HLC_INT)
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	0	0	0
0	0	1	1	0	1	0
0	1	0	0	0	0	0
0	1	0	1	0	0	0
0	1	1	0	0	1	0
0	1	1	1	1	0	0
1	0	0	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	0	1	1	1	1	0
1	1	0	0	0	0	0
1	1	0	1	0	0	0
1	1	1	0	1	1	0
1	1	1	1	0	0	1

逻辑方程如下:

- S1 = !S1&S0&E1&E0 | S1&!S0&E1&!E0 | S1&!S0&E1&E0 | S1&S0&E1&!E0
- S0 = !S1&!S0&E1&E0 | !S1&S0&E1&!E0 | S1&!S0&E1&E0 | S1&S0&E1&!E0
- OUTPUT = S1&S0&E1&E0

4.5.3 时钟配置

CCSI 总线通信所需的时钟由片上 PWM 生成。PWM*n*A 输出生成频率为目标 SCLK 2 倍的 PWM_SCLKX2 时钟，而 PWM*n*B 输出生成所需频率的 PWM_SCLK 时钟。

TX 和 RX CLB Tile 始终使用 PWM_SCLKX2 时钟发送和接收数据。系统中只有 PWM_SCLK 最终连接到 LED 驱动器。

通过配置 TX Tile 的 PWM_SCLK 输入源，可实现单时钟边沿或双时钟边沿的数据收发。若需单时钟边沿收发（例如使用 LP5891-Q1 LED 驱动器时），可修改 TX Tile 输入配置，将 PWM*n*A 输出同时接到 PWM_SCLK 和 PWM_SCLKX2 输入，见图 4-12。PWM*n*B 输出可不连接到 CLB Tile。

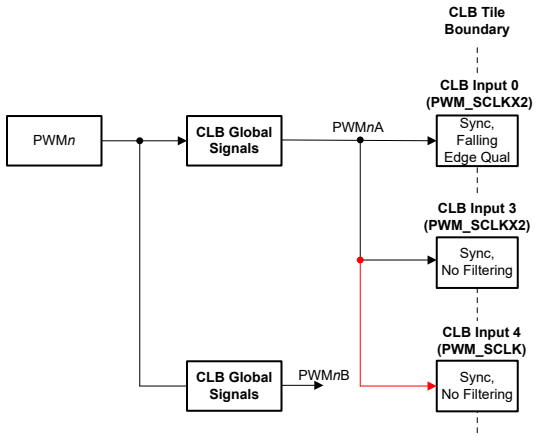


图 4-12. 单时钟边沿收发的时钟配置

Copyright © 2023 Texas Instruments Incorporated

双时钟边沿收发支持在时钟的两个边沿进行发送和接收，见图 4-13。

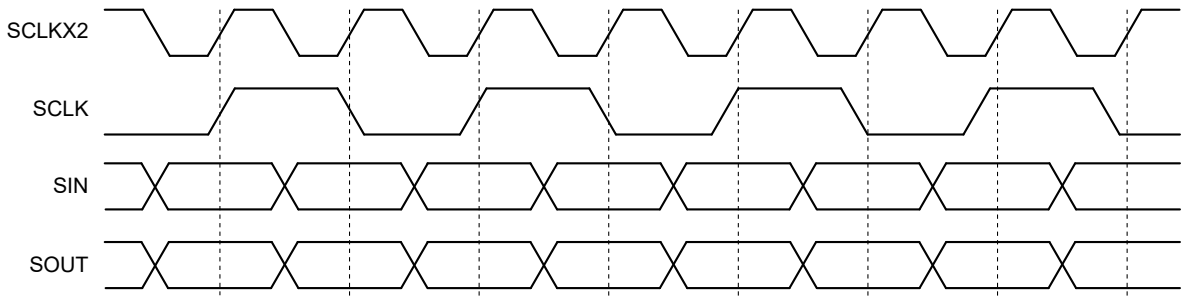


图 4-13. 使用 PWM_SCLKX2 的双时钟边沿收发

若需双时钟边沿收发，可修改 TX Tile 输入配置，将 PWM*n*A 输出接到 PWM_SCLKX2 输入，并将 PWM*n*B 输出接到 PWM_SCLK 输入，见图 4-14。

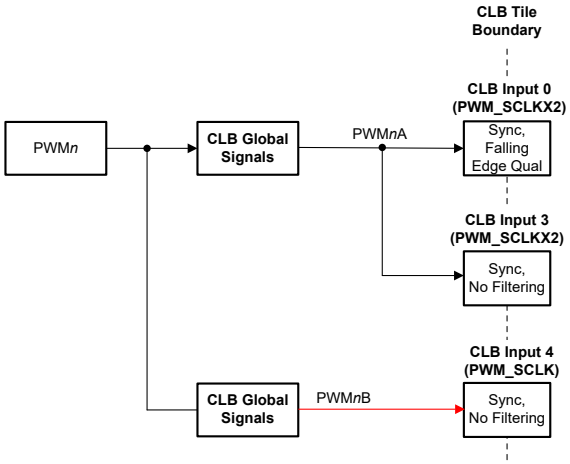


图 4-14. 双时钟边沿收发的时钟配置

为减少 CLB 逻辑引入的时序延迟，两个 PWM 时钟输入都会穿过 TX CLB 逻辑。

Copyright © 2023 Texas Instruments Incorporated

4.6 步骤 5：仿真逻辑设计

图 4-15 展示了数据接收仿真。仿真使用简单的数据输入模式 0xAAAA，展示帧起始（由 START 位标识）、完整接收字的捕获以及 CHECK 位的验证过程。

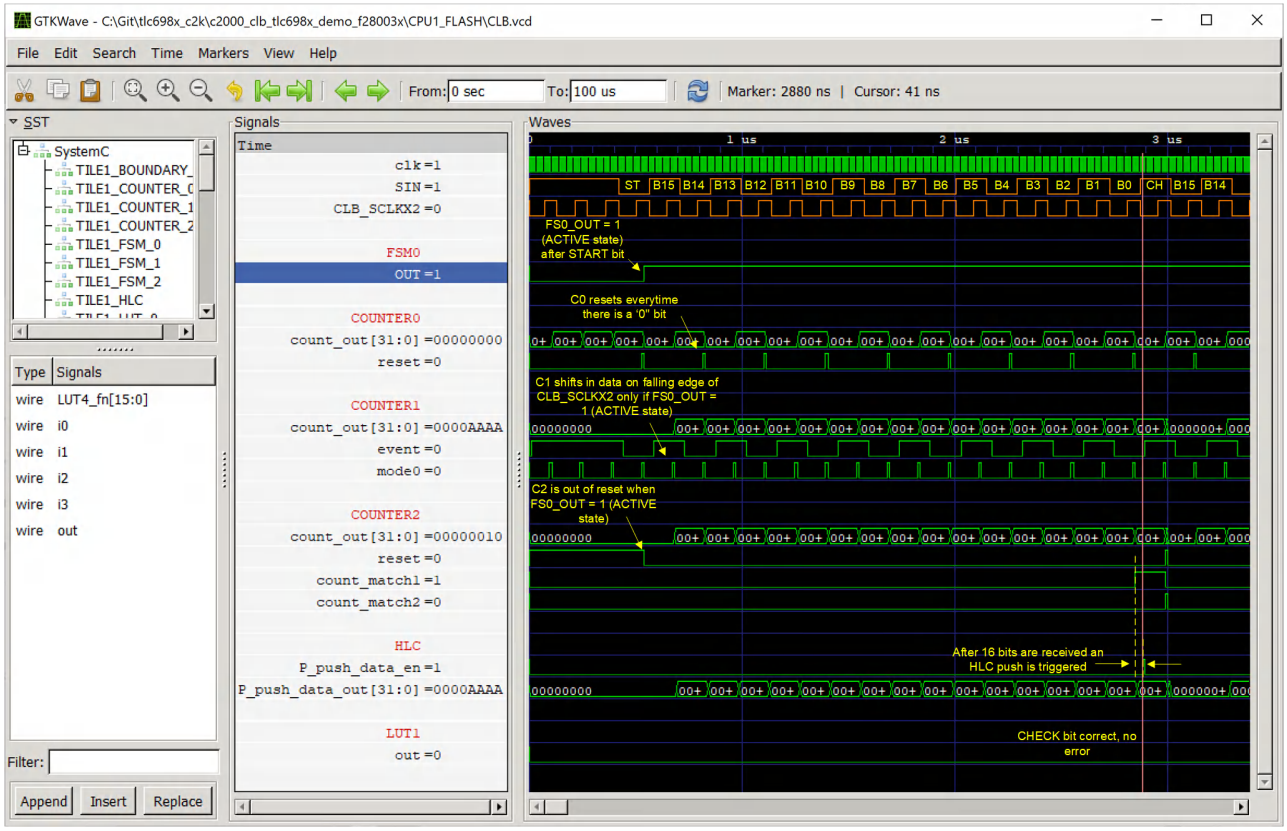


图 4-15. LED 驱动器的数据接收仿真

Copyright © 2023 Texas Instruments Incorporated

图 4-16 展示了另一组数据接收仿真。该仿真在输入数据流中加入错误的 CHECK 位，以验证 CHECK 位逻辑是否正常工作。仿真显示 LUT1 模块输出变为高电平，表示检测到 CHECK 位错误。

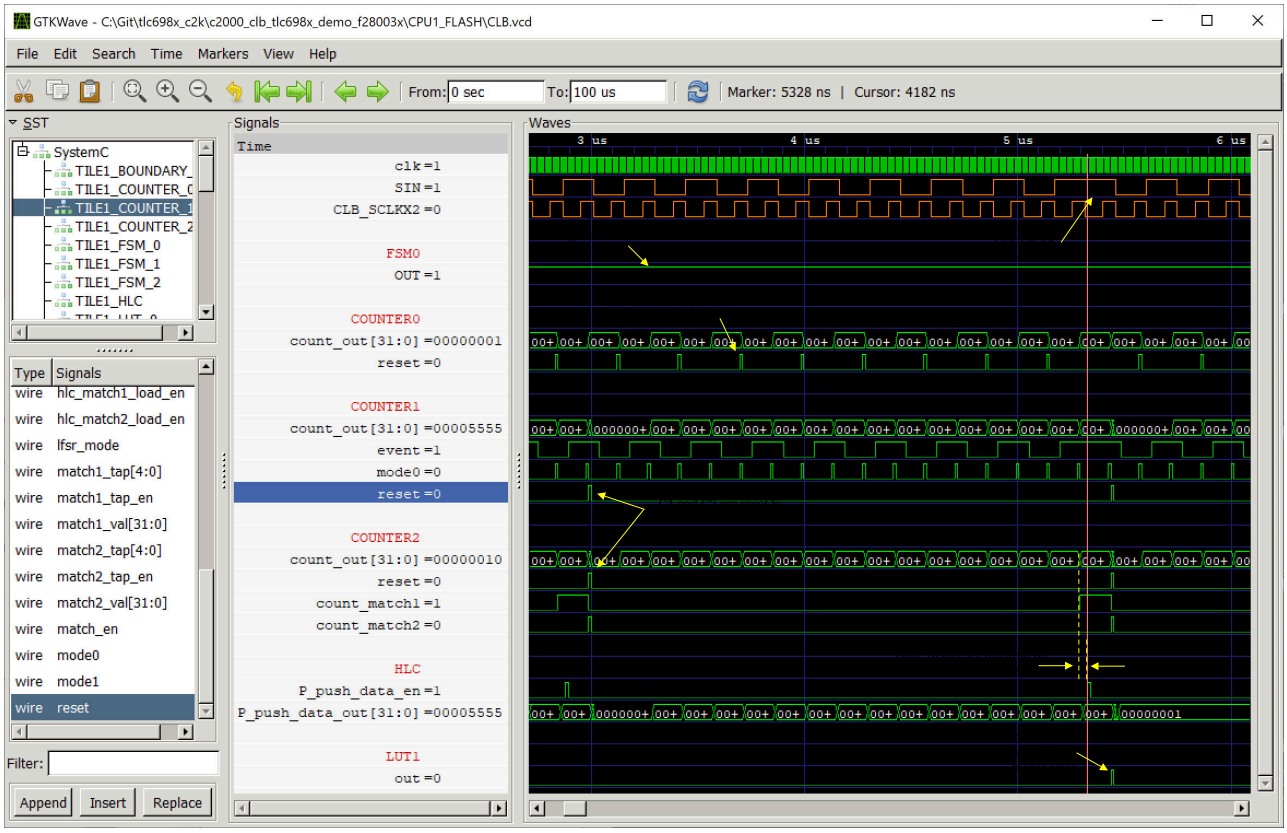


图 4-16. 校验位错误逻辑仿真

最后，图 4-17 展示了 END 位的检测过程。检测到 END 位后，FSM0 模块会使 CLB 逻辑切换到 IDLE 状态。

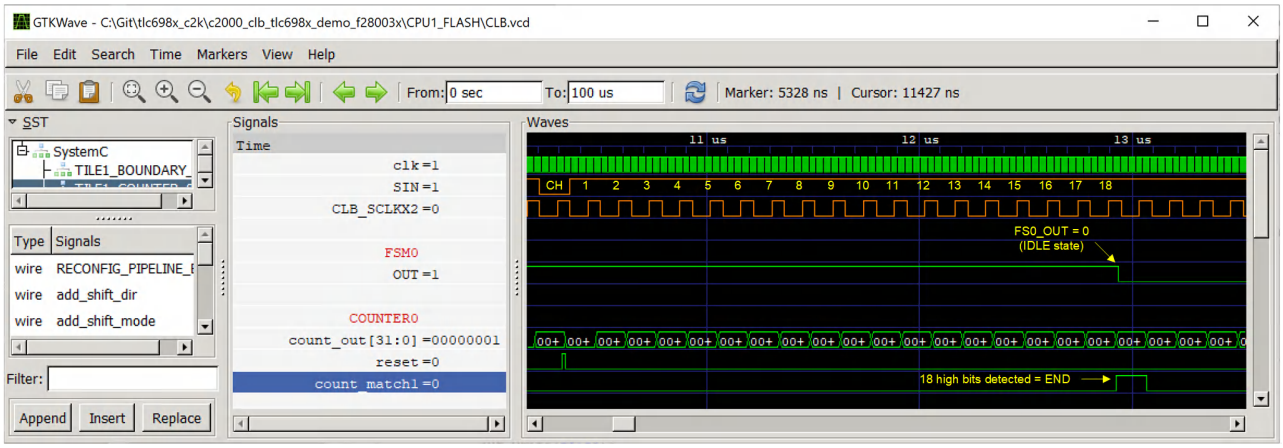


图 4-17. END 帧检测

注：接收中断和 CHECK 位错误（未显示）都会在 END 位期间产生。CPU 代码必须始终丢弃帧中的最后一个字，因为该字对应 END 位。

Example B: Using the CLB to I

Copyright © 2023 Texas Instruments Incorporated

图 4-18 展示了数据发送仿真。仿真从 START 位开始，发送一个 0x5555 字，最后以 END 帧结束。为简化 CLB 逻辑，Tile 通过发送 0xFFFF 字来实现 START 位，并使用 Tile 的 CHECK 位逻辑在第 17 个时钟周期生成 START 位。生成 END 帧时，CLB 逻辑只需在最后一个字发送完毕后将 CLB_SOUT 置为 1。开始下一次数据发送前，CLB 逻辑依赖 CPU 至少等待 18 个 PWM_SCLKX2 周期。

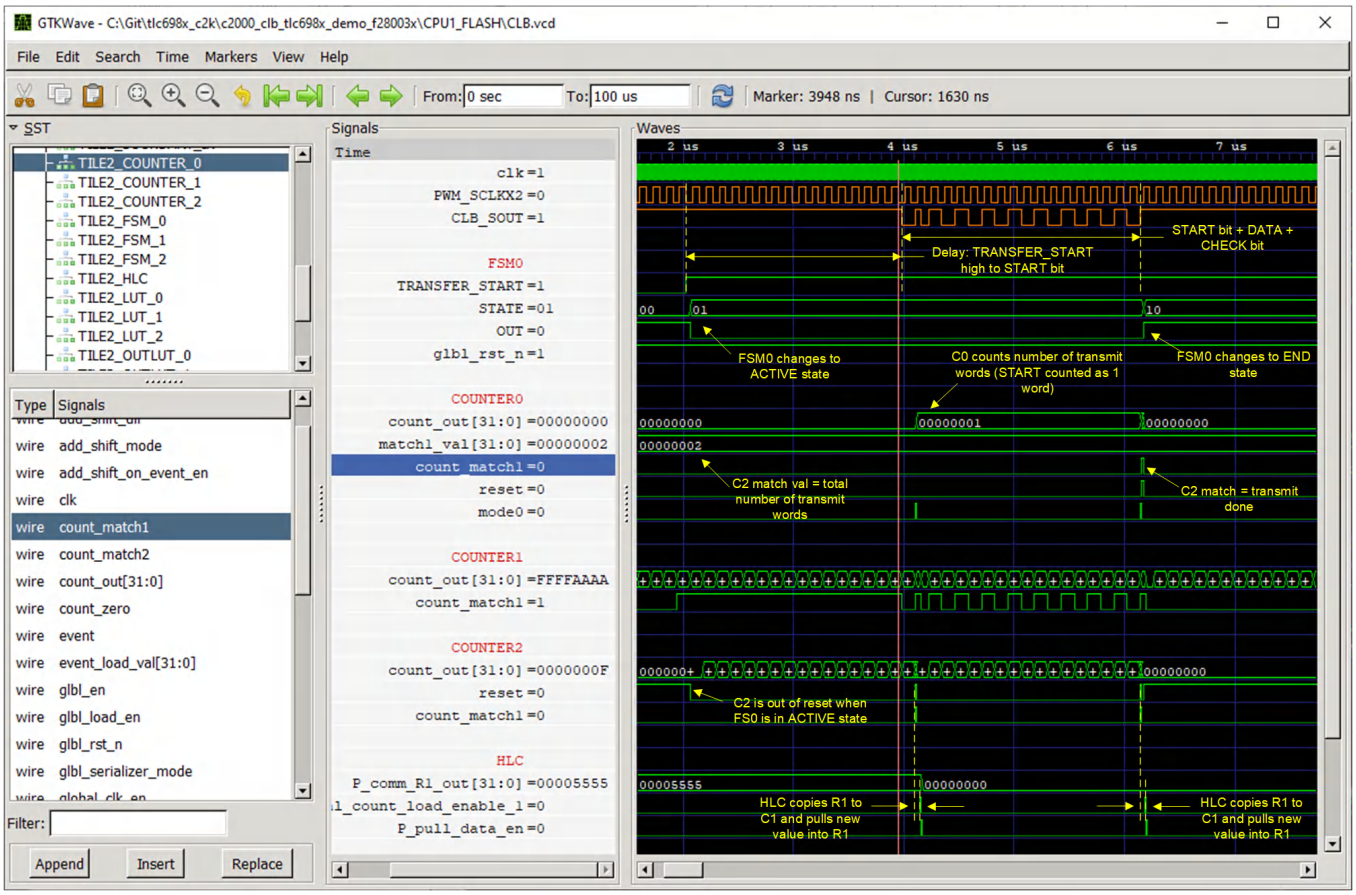


图 4-18. LED 驱动器数据发送仿真

Copyright © 2023 Texas Instruments Incorporated

数据发送期间，由于输出串行器（Counter 1）更新延迟，会出现输出毛刺。图 4-19 显示了从 PWM_SCLKX2 下降沿到输出串行器最终更新之间的内部延迟。毛刺会缩短接收器件的建立时间。由于 PWM_SCLKX2 时钟经过 CLB Tile（TILE）时也会产生延迟，因此不应以该信号的上升沿作为建立时间的基准。应改用 CLB_SCLKX2 信号计算建立时间；接收器件看到的正是这个延迟后的 PWM_SCLKX2 信号。

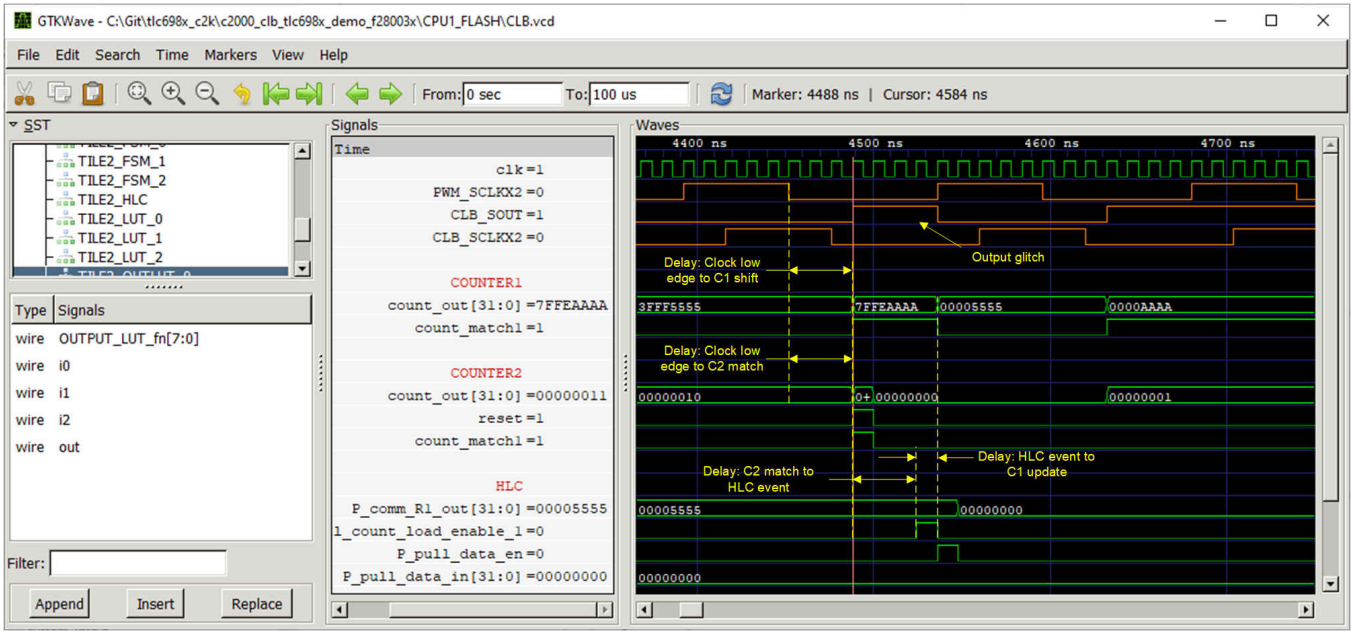


图 4-19. 输出毛刺

将串行器输出锁存后再送至 CLB_SOUT，可消除毛刺。不过，本例未采用此方法，因为发送仿真表明至少可获得 2 个 CLB 周期的建立时间，足以满足 LP5891-Q1 器件所需的 10ns 建立时间。

4.7 步骤 6：测试 CLB 逻辑

本节介绍如何测试 LED 驱动器示例附带的 Code Composer Studio™ 项目。该示例已在 F280039C LaunchPad（LAUNCHXL-F280039C）上进行测试。

测试中使用一个 LP5891-Q1 EVM，从 F280039C MCU 接收并显示图像数据 . LP5891-Q1 EVM 配有 16x16 矩阵 LED 显示屏。

4.7.1 硬件设置和连接

运行演示所需的硬件如下：

- 1x LAUNCHXL-F280039C LaunchPad
- 1x LP5891Q1EVM 矩阵 LED 显示屏

可选设备

- 逻辑分析仪（用于查看 CCSI 总线信号）

按以下步骤连接硬件：

1. 按默认设置配置 LAUNCHXL-F280039C EVM：
   a. 安装 JP1 跳线，使 USB-C 接口提供的 5 V 电源和 GND 连接到电路板的 XDS110 一侧。b. 安装 JP2 跳线，将 XDS110 一侧的 5 V 电源连接到 LaunchPad 的其余部分。c. 在 J101 上安装 TCK 和 TMS 跳线，将 XDS110 连接到 F280039C 器件（也可按需安装 J101 上的其他跳线）。d. S3 引导模式设为 Flash boot（可选）。

Copyright © 2023 Texas Instruments Incorporated

2. 按以下步骤配置 LP5891Q1EVM：

a. 将开关 S1 设为 SOUTHOST，使 LP5891-Q1 器件的串行数据输出返回主机控制器。b. 将跳线 J6 设为 MCU5 V，选择主机控制器提供的 5 V 电源。c. 将跳线 J7 设为“3V3 VR”，为 VLEDR 选择 3.3 V 电源。

3. 将 LP5891Q1EVM 插入 LAUNCHXL-F280039C EVM 的位置 1。参照两块板上的 5 V、3V3 和 GND 标记，确保板卡方向正确。4. 将 USB-C 线缆连接到 LAUNCHXL-F280039C EVM 的 USB 接口。将线缆插入 USB2.0/USB3.x 端口后，LaunchPad 和 LP5891Q1EVM 会由 USB 端口供电。5. 继续进行软件设置。

4.7.2 软件设置

构建和运行软件需要以下软件包：

- Code Composer Studio（CCS）11.1.0 版或更高版本
- C2000Ware 4.02.00.00 版或更高版本

（可选）GNU 编译器（TDM-GCC）和用于运行仿真的 GTK Wave 仿真查看器。更多信息请参阅 *CLB Tool's User's Guide*。

按照以下步骤在 CCS 中配置 LED 示例软件：

1. 在 CCS 菜单中单击“Project -> Import CCS Projects...”。2. 在“Select search-directory”中输入 F 28003x DriverLib CLB 示例项目路径。a. 路径：<C2000WARE_INSTALL>\driverlib\f28003x\exampl es\clb 3. 单击“Refresh”。4. 选择“clb_ex32_led_driver”项目。

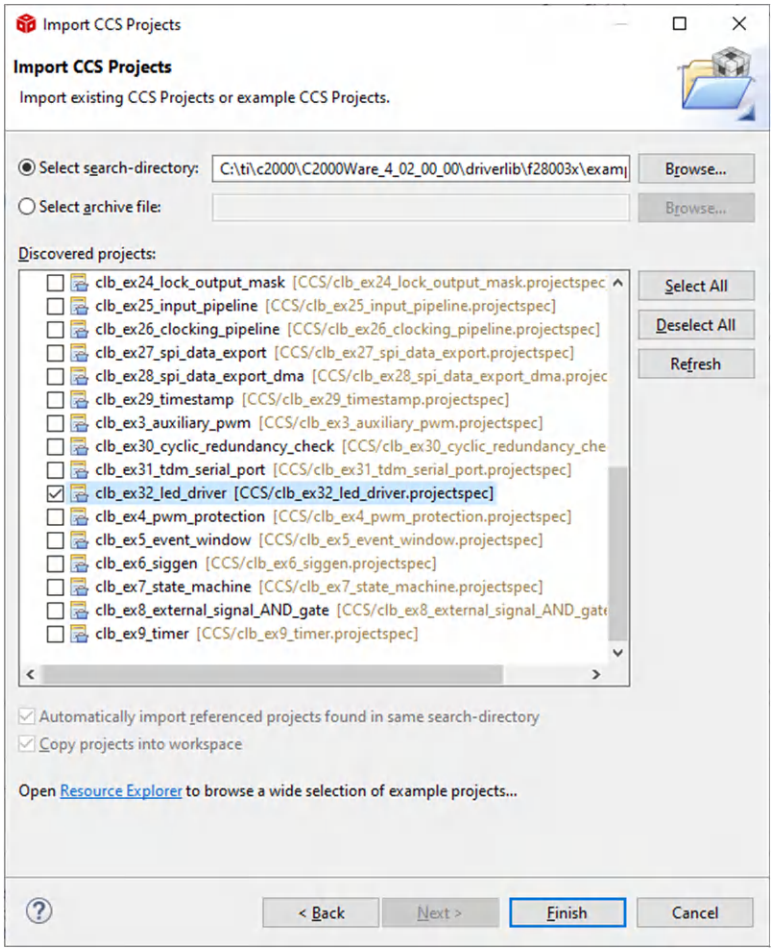


图 4-20. 导入 CLB LED 驱动器示例项目

Copyright © 2023 Texas Instruments Incorporated

5. 勾选“Copy projects into workspace”。6. 单击“Finish”，将项目导入工作区。可选：单击“Project -> Build Configurations -> Set Active -> CPU1_LAUNCHXL_FLASH”，以构建用于 Flash 运行的代码。7. 在 CCS 菜单中单击“Project -> Build Project”，构建示例项目。8. 在项目中右键单击 TMS320F280039C_LaunchPad.ccxml 文件，并选择“Set as Active Target Configuration”。9. 单击“Run -> Debug”，将可执行文件加载到 F280039C 目标器件。10. 最后，在 CCS 的 Debug 透视图中单击“Run -> Resume”以运行代码。

4.7.3 测试输出建立时间和保持时间

CLB 输出 CCSI 总线的一个关键考量是接收器件所见的建立时间和保持时间。为测量这两个时间，使用示波器持续采集 CLB 输出。

如图 4-21 所示，CLB 输出的最小建立时间约为 55 ns，远大于 LP5891-Q1 LED 驱动器所需的 10 ns。请注意 CLB_SOUT_1 输出的变化。正如第 4.6 节所述，这是串行器输出更新时内部延迟变化所致。将串行器输出锁存后再送至 CLB_SOUT，可消除毛刺。不过，由于最终输出的建立时间已满足 LP5891-Q1 LED 驱动器的要求，因此本例未采用这种方法。

保持时间至少为半个 SCLK 周期。

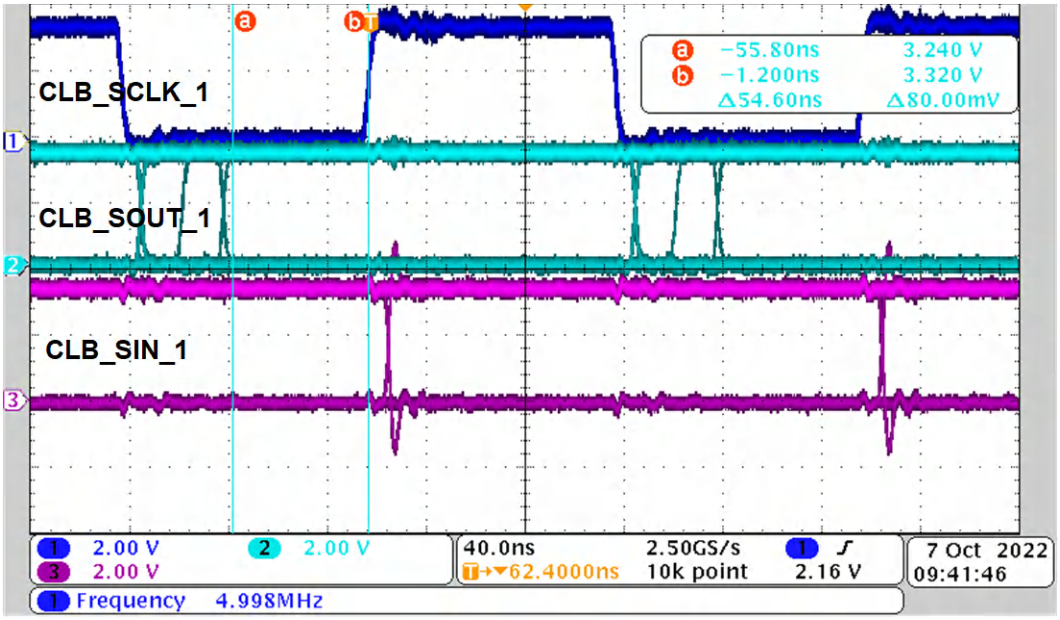


图 4-21. LED 驱动器示例的建立时间和保持时间

Copyright © 2023 Texas Instruments Incorporated

5 参考资料

- 可配置逻辑块（CLB）简介（视频）
- 可配置逻辑块（CLB）架构（视频）
- 可配置逻辑块（CLB）编程工具培训（视频）
- *CLB Tool User's Guide*
- *TMS320F28388D controlCARD Information Guide*
- 面向 C2000 实时控制微控制器的 SysConfig 开发工具
- C2000 Academy 在线培训
- Code Composer Studio
- C2000Ware
- TMDSCNCD280025C
- TMDSCNCD28388D
- TMDSHSECDOCK
- LAUNCHXL-F280039C
- LP5891Q1EVM

Copyright © 2023 Texas Instruments Incorporated

重要声明与免责声明

TI 按“原样”提供技术和可靠性数据（包括数据手册）、设计资源（包括参考设计）、应用或其他设计建议、Web 工具、安全信息及其他资源，并不保证其不存在任何缺陷。TI 明确拒绝所有明示和默示保证，包括但不限于对适销性、特定用途适用性以及不侵犯第三方知识产权的默示保证。

这些资源面向使用 TI 产品进行设计的专业开发人员。您应独自负责：（1）为应用选择合适的 TI 产品；（2）设计、验证和测试应用；（3）确保应用符合适用标准以及其他安全、安保、法规或相关要求。

这些资源如有变更，恕不另行通知。TI 仅允许您将这些资源用于开发使用资源中所述 TI 产品的应用。禁止以其他方式复制或展示这些资源。TI 未授予您任何其他 TI 知识产权或第三方知识产权的许可。对于您使用这些资源而引起的任何索赔、损害、成本、损失和责任，TI 不承担责任；您应向 TI 及其代表提供充分赔偿，使其免受上述责任。

TI 产品的销售受 TI 销售条款或其他适用条款约束；相关条款可在 ti.com 上查看，或随 TI 产品一并提供。TI 提供这些资源不会扩大或以其他方式更改适用于 TI 产品的保证或免责声明。

TI 反对并拒绝您提出的任何附加条款或不同条款。 重要声明

Mailing Address: Texas Instruments, Post Office Box 655303, Dallas, Texas 75265
Copyright © 2023, Texas Instruments Incorporated